Pontificia Universidad Católica del Perú
Facultad de Ciencias e Ingeniería
Departamento Académico de Ingeniería

1INF02 APRENDIZAJE DE MÁQUINA

Redes Neuronales

Prof. Rodrigo Uribe

Organización de las Clases

(10 min) Preguntas y consultas.

(30 min) Respuesta *Vibe Quest.*

(35 min) Redes neuronales de múltiples capas como una forma de aprender características.



(10 min) Break

(45 min) Intuición general de arquitecturas de redes neuronales.

(20 min) Ejercicio básicos.

(10 min) Preguntas *Vibe Quest.*



Respuestas Vibe Quest

- ¿En qué situaciones maximizar la varianza (el objetivo de PCA) podría llevarnos a perder información crucial para nuestro problema específico?
- ¿Qué tipos de relaciones no lineales o estructuras topológicas complejas se pierden inevitablemente en PCA, y cómo podríamos detectar que esta pérdida es significativa para nuestro análisis?
- ¿Hasta qué punto podemos atribuir significado real a los componentes principales, y qué peligros existen al interpretar estas construcciones matemáticas como fenómenos causales subyacentes?
- ¿De qué manera el Análisis de Componentes Principales representa una extensión natural de conceptos estadísticos clásicos como la varianza y correlación, y en qué sentido constituye una ruptura filosófica con los paradigmas tradicionales de inferencia estadística?

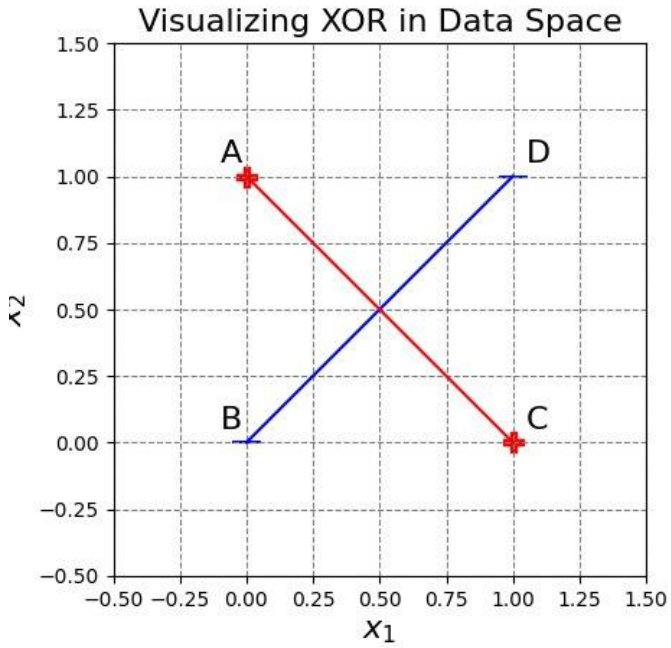
¿Qué modelos hemos visto hasta ahora?

¿Qué tipo de modelo usa PCA?

XOR NO es linealmente separable

Las neuronas individuales son clasificadores lineales y tienen un poder expresivo limitado. Por ejemplo, la función XOR no es linealmente separable (se puede demostrar mediante convexidad).

	x_1	x_2	t
A	0	1	1
B	0	0	0
C	1	0	1
D	1	1	0

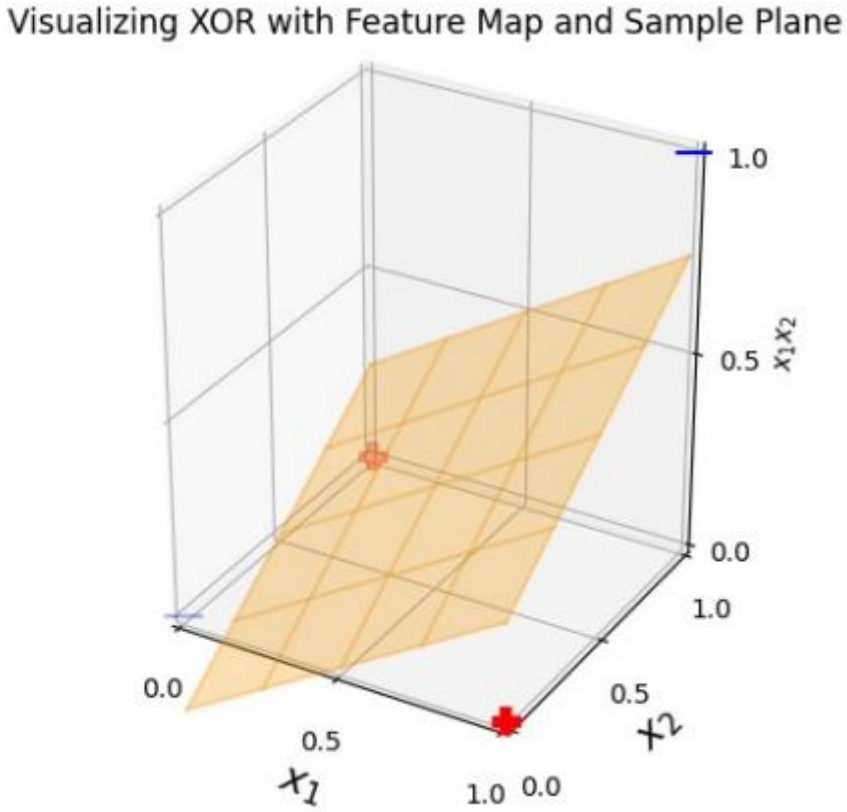


¿Qué truco podemos usar?

Clasificar XOR mediante mapas de características

A veces, podemos utilizar mapas de características para superar esta limitación.

	x_1	x_2	x_1x_2	t
A	0	1	0	1
B	0	0	0	0
C	1	0	0	1
D	1	1	1	0



Un ejemplo diferente: primera labor como analista de datos

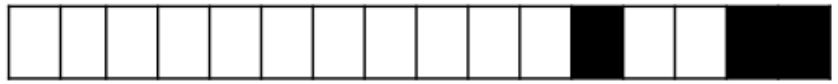
Consideremos dos vectores A y B de 16 dimensiones en todas las traslaciones.

¿Podemos construir un clasificador lineal para distinguirlos?

Patrón A:



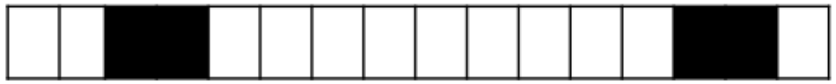




Patrón B:







¡La invariancia de la traslación es deseable en la visión!

No linealmente separable

¿Podemos construir un clasificador lineal para distinguir A y B en todas las traslaciones posibles?

Patrón A



Patrón B



- Todas las traslaciones del Patrón A deben clasificarse como "A", y todas las traslaciones del Patrón B deben clasificarse como "B“
- Si todas las traslaciones de A → clase A, entonces su promedio también → clase A (esto es obligatorio para clasificadores lineales)
- El promedio de todas las traslaciones de A = (0.25, 0.25, ..., 0.25) y el promedio de todas las traslaciones de B = (0.25, 0.25, ..., 0.25) - ¡Son idénticos!
- La contradicción inevitable: El vector (0.25, 0.25, ..., 0.25) debe clasificarse como "A" y como "B" al mismo tiempo → ¡Imposible! Por lo tanto, no puede existir tal clasificador lineal

Superando los límites de la clasificación lineal

En general, es difícil crear nuevas características útiles.

¿Cómo usar un modelo para aprender hipótesis no lineales directamente?

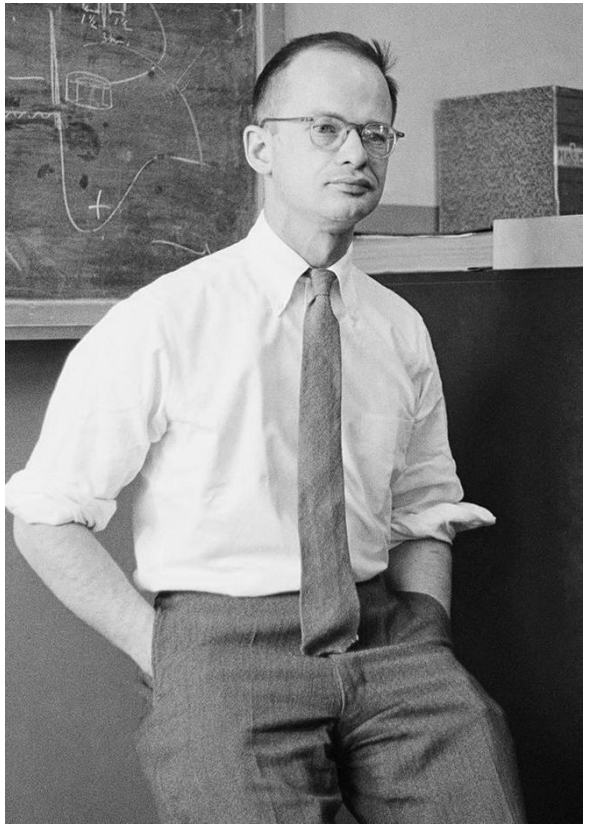


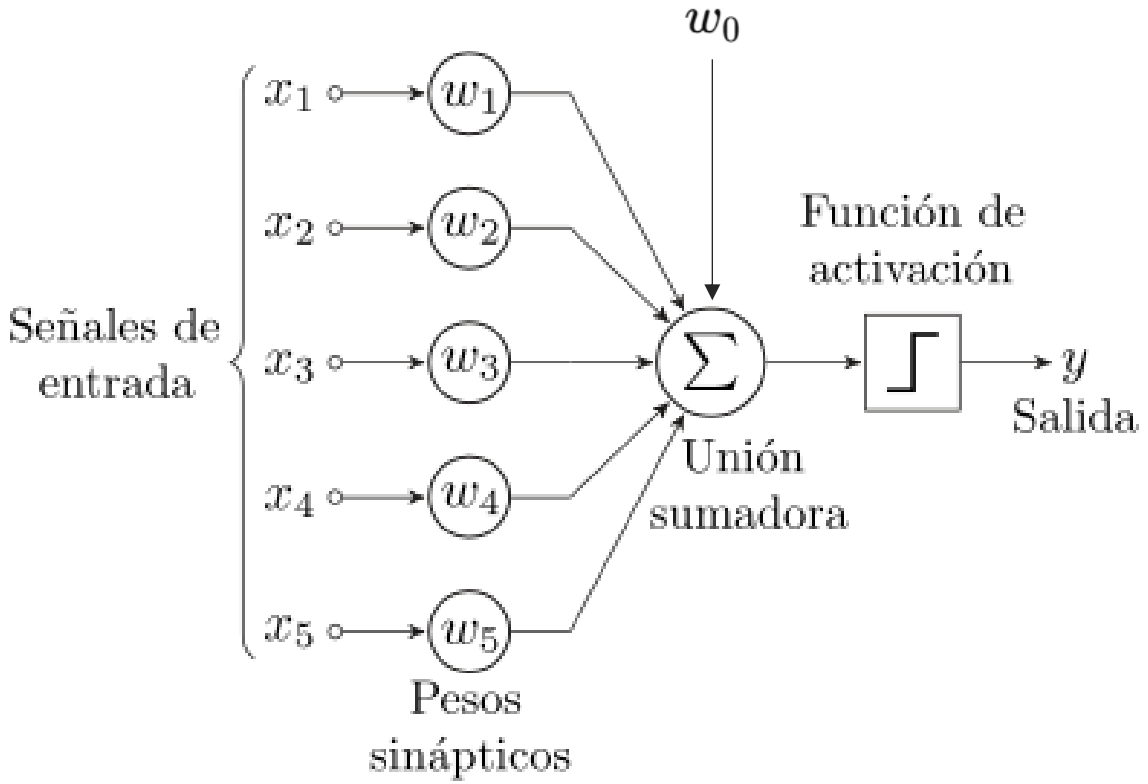
¿Qué modelo no lineal hemos aprendido?

¿Cuál era el problema del perceptrón?

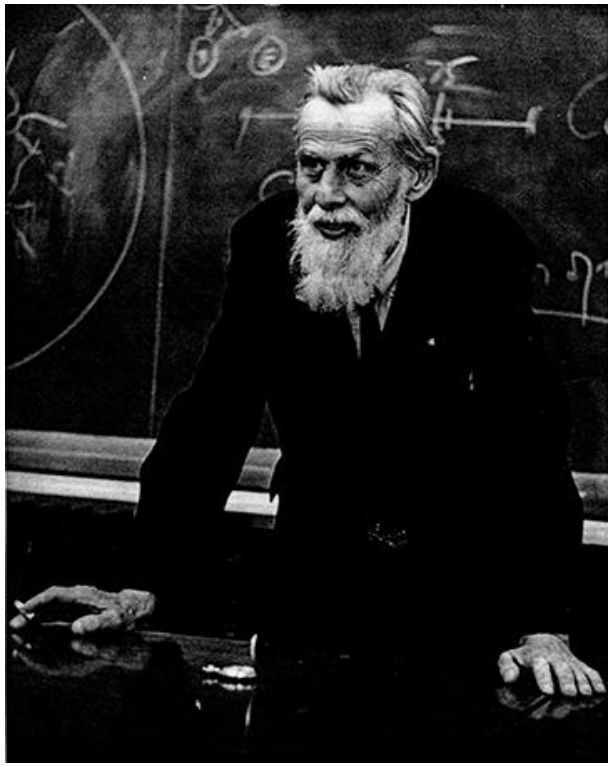
El Perceptrón (1943)

Walter Pitts

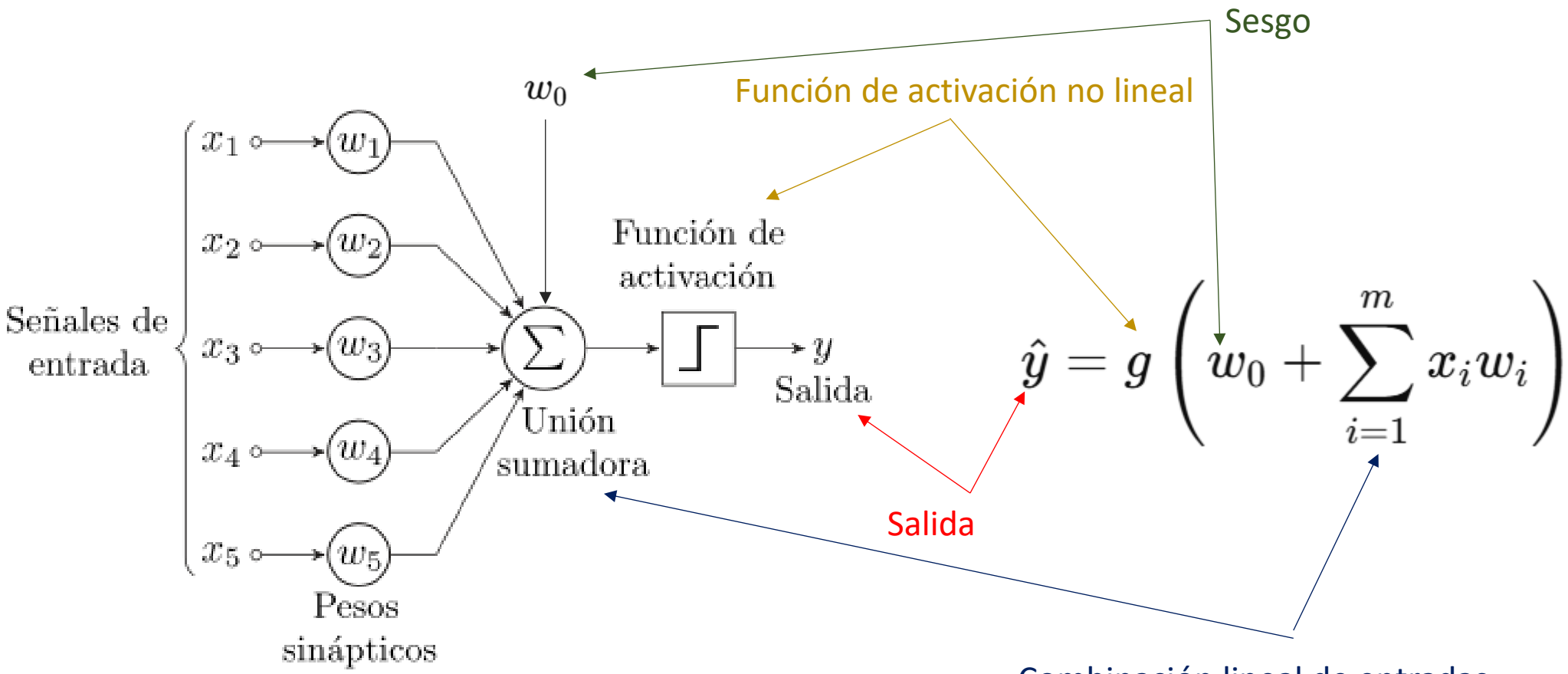




Warren McCulloch



El Perceptrón (1943)



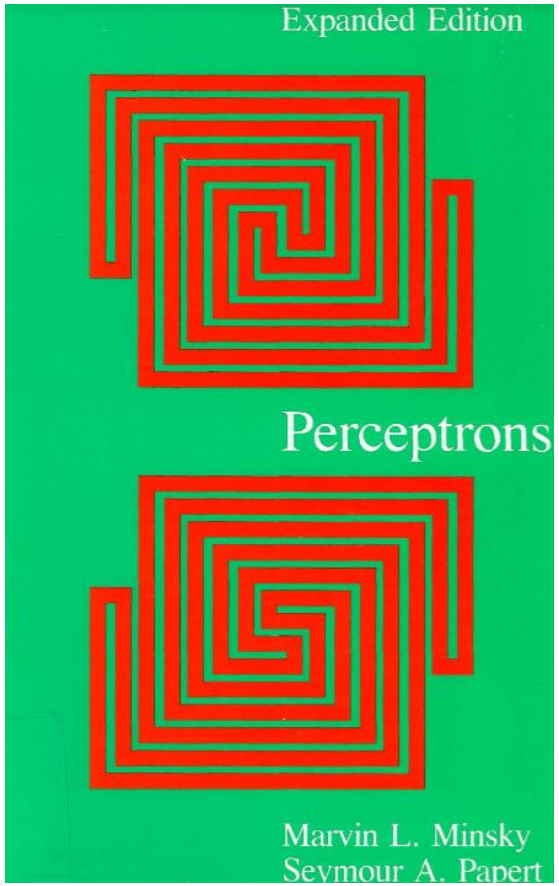
- Se puede ver la regresión logística como una sola unidad.
- Una vez que conectamos muchas unidades en una red, se vuelven mucho más poderosas.

El Perceptrón como una generalización simplificada de clasificación de modelos lineales

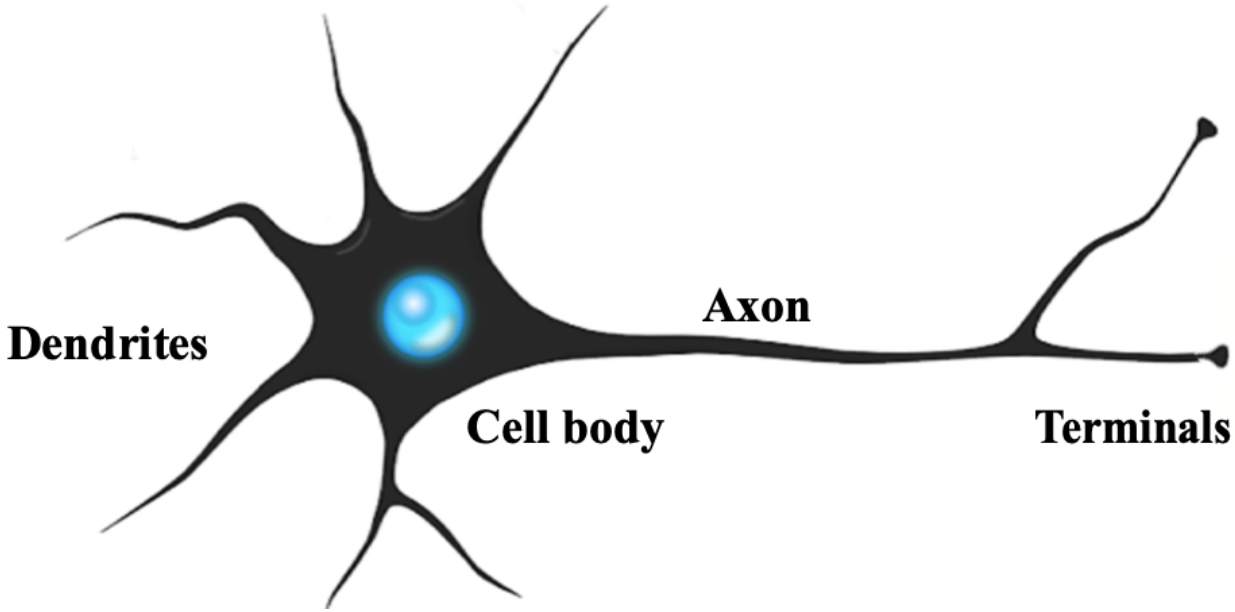
	Logistic Regression	Softmax Regression
Linear Model	$z = \mathbf{w}^T\mathbf{x}$	$\mathbf{z} = \mathbf{W}\mathbf{x}$
Activation	$y = \sigma(z) = \frac{1}{1+e^{-z}}$	$\mathbf{y} = \text{softmax}(\mathbf{z})$ $y_k = \frac{e^{z_k}}{\sum_{m=1}^{K} e^{z_m}}$
Cross-Entropy Loss	$L_{CE}(y,t)$ $= -t \log y$ $-(1-t)\log(1-y)$	$L_{CE}(\mathbf{y},\mathbf{t}) = -\mathbf{t}^T \log(\mathbf{y})$ $= -\sum_{k=1}^{K} t_k \log(y_k)$

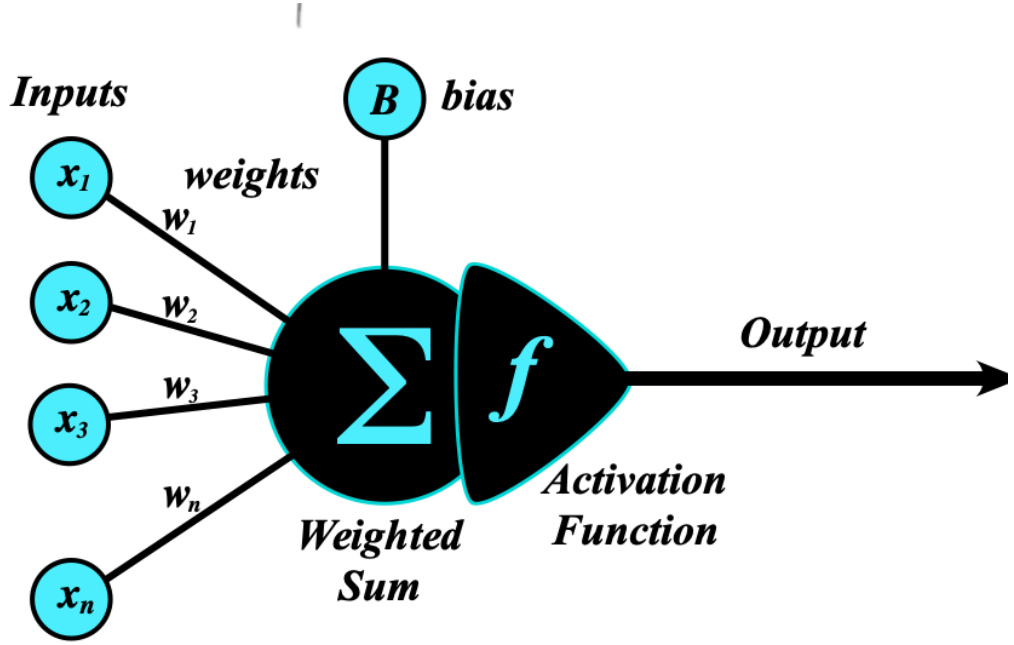
Primera Invierno de la IA: Limitaciones del Perceptrón

- En 1969, Minsky y Papert demostraron que el perceptrón no podía resolver problemas no lineales como XOR.
- Esto expuso graves limitaciones en las redes neuronales de una sola capa.
- Causó una pérdida de interés y financiamiento, iniciando el primer invierno de la inteligencia artificial.

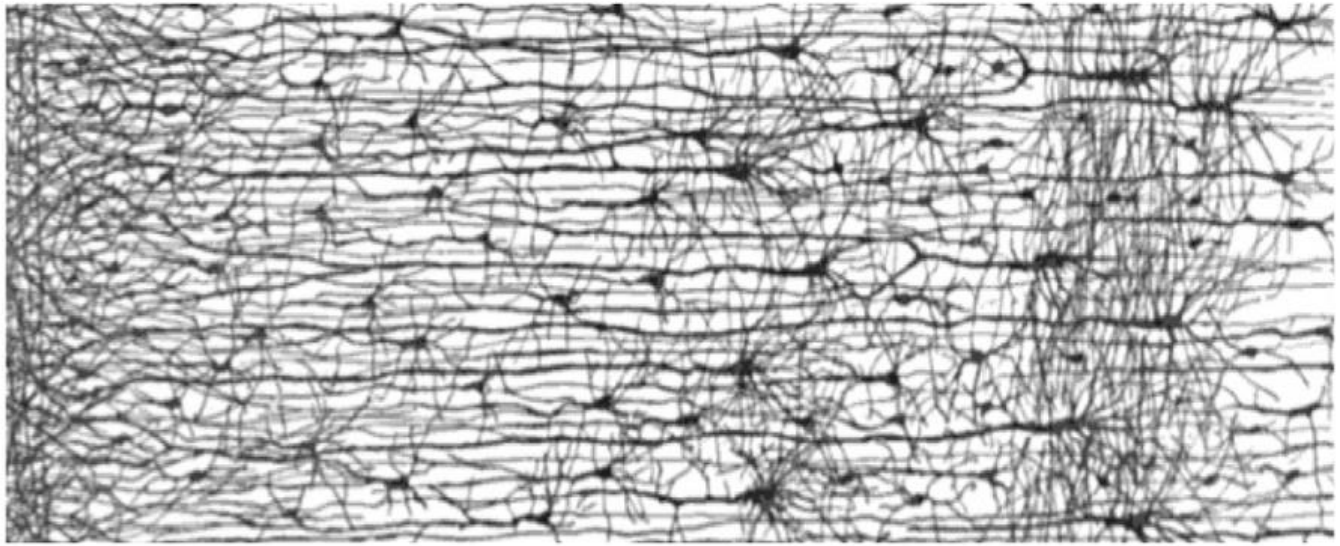


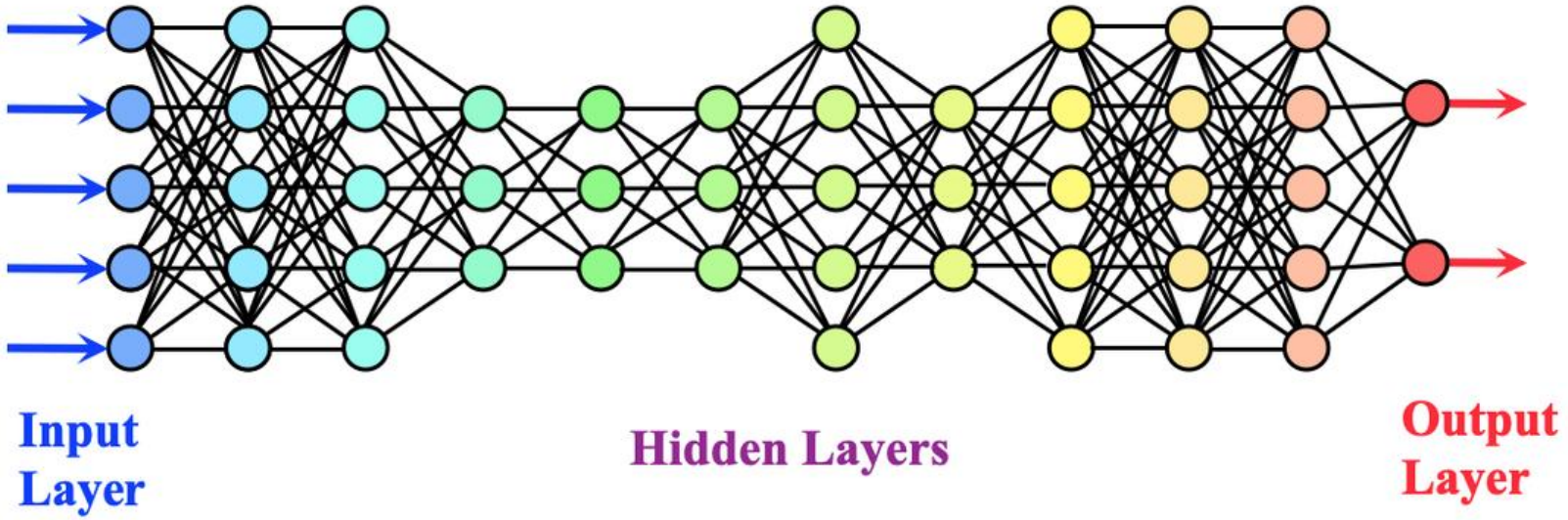
El Perceptrón fue inspirado en una neurona



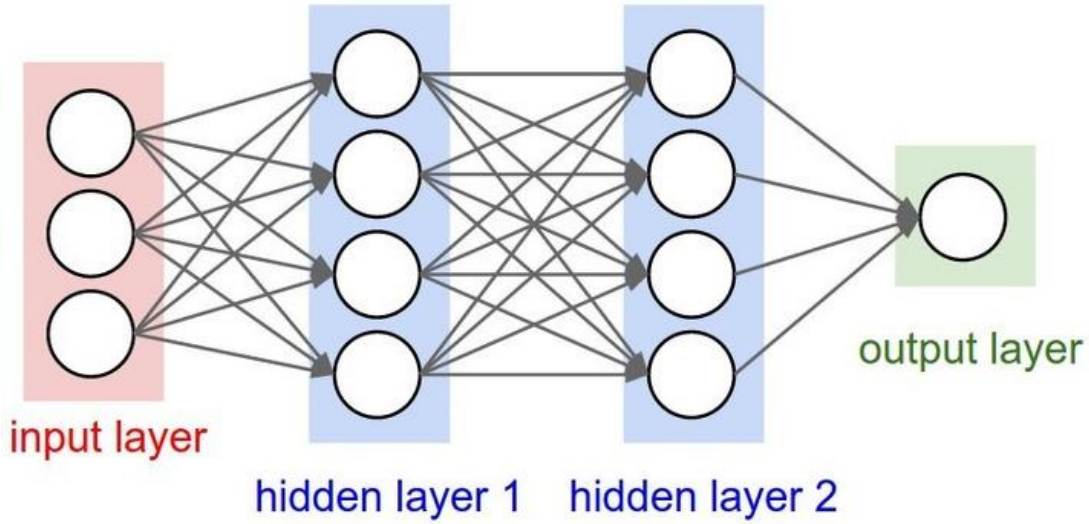


Inspirado en las neuronas reales...





Perceptrones multicapa (MLP)



Capas:

Capa de entrada: introduce las características de entrada.

Capas ocultas: calcula características avanzadas.

Capa de salida: salida interpretable.

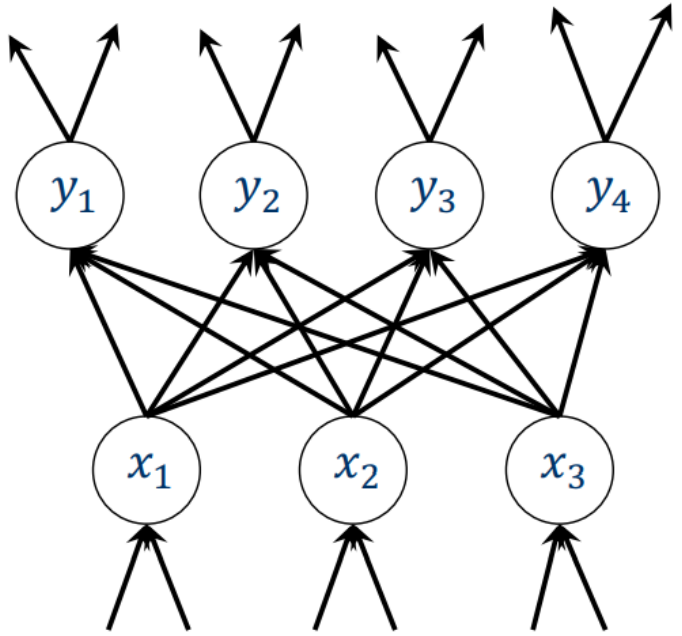
Una capa completamente conectada

Cada unidad de entrada se conecta a cada unidad de salida.

N input units, M output units

$M \times N$ weight matrix

$$y = \phi(Wx + b)$$

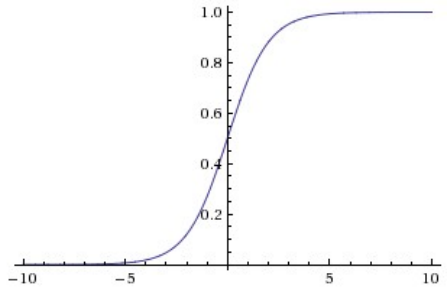


Perceptrón multicapa: una red multicapa que consta de capas completamente conectadas.

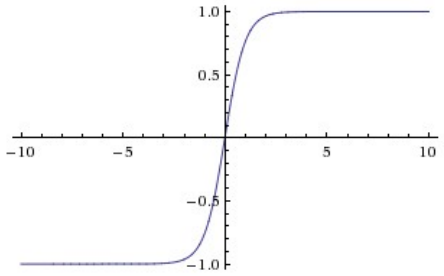
Funciones de Activación

Sigmoid

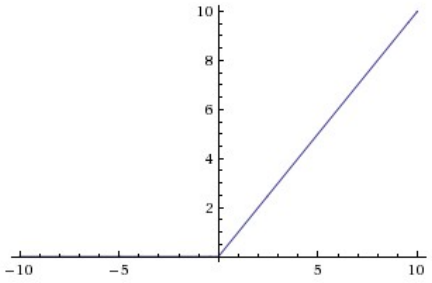
$$\sigma(x) = 1/(1 + e^{-x})$$



tanh tanh(x)

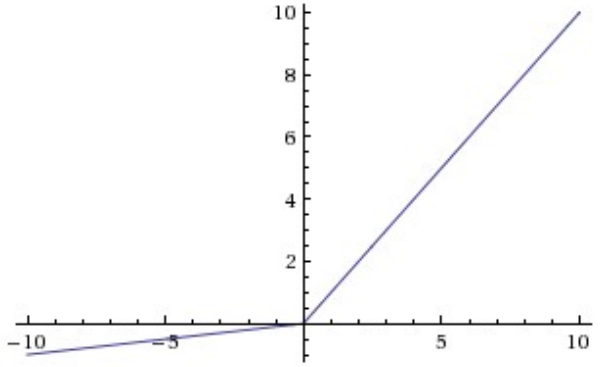


ReLU max(0,x)



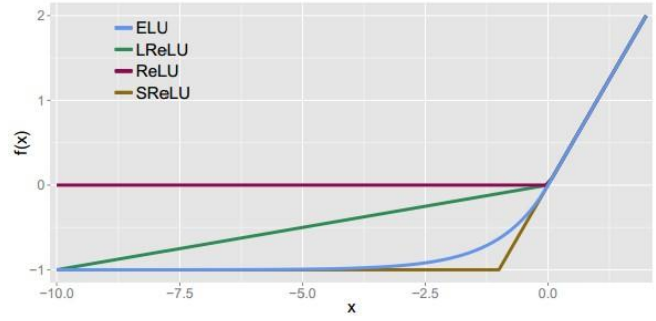
Leaky ReLU

max(0.1x, x)

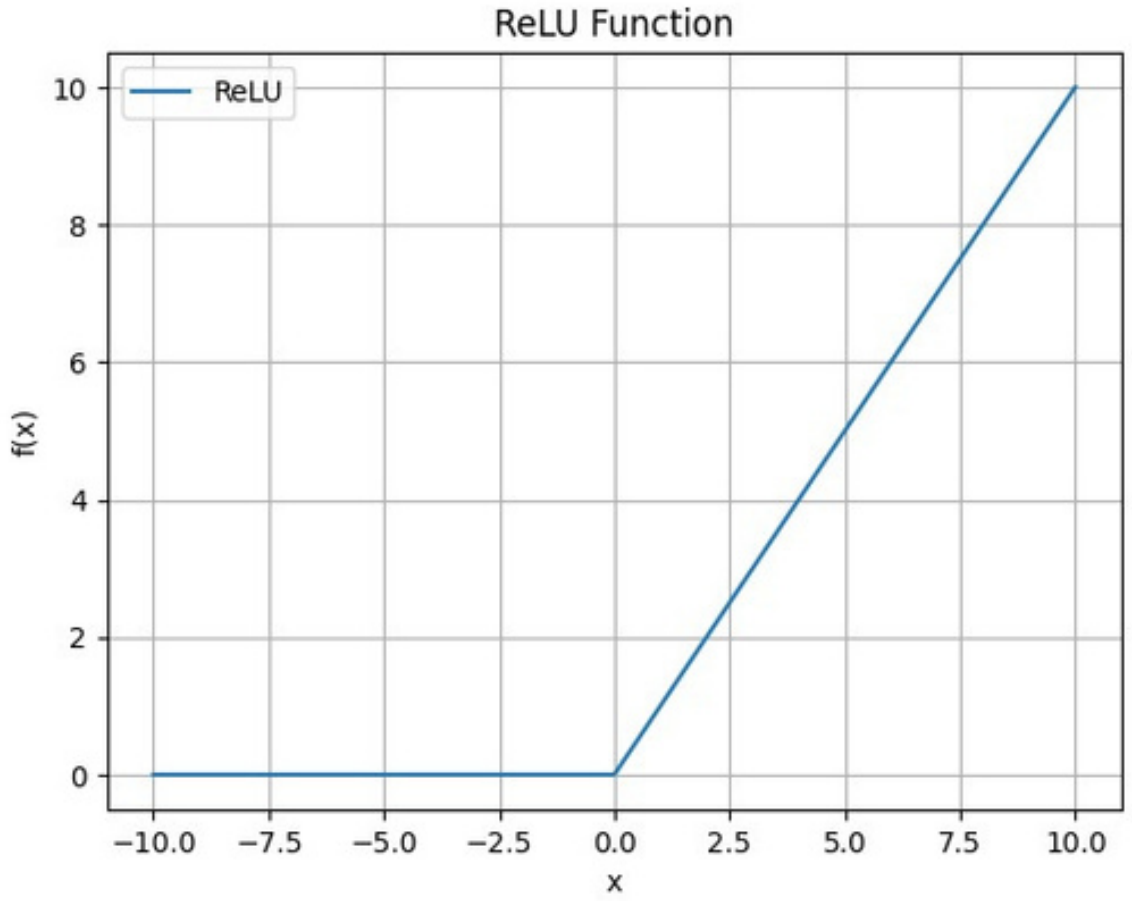


ELU

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$



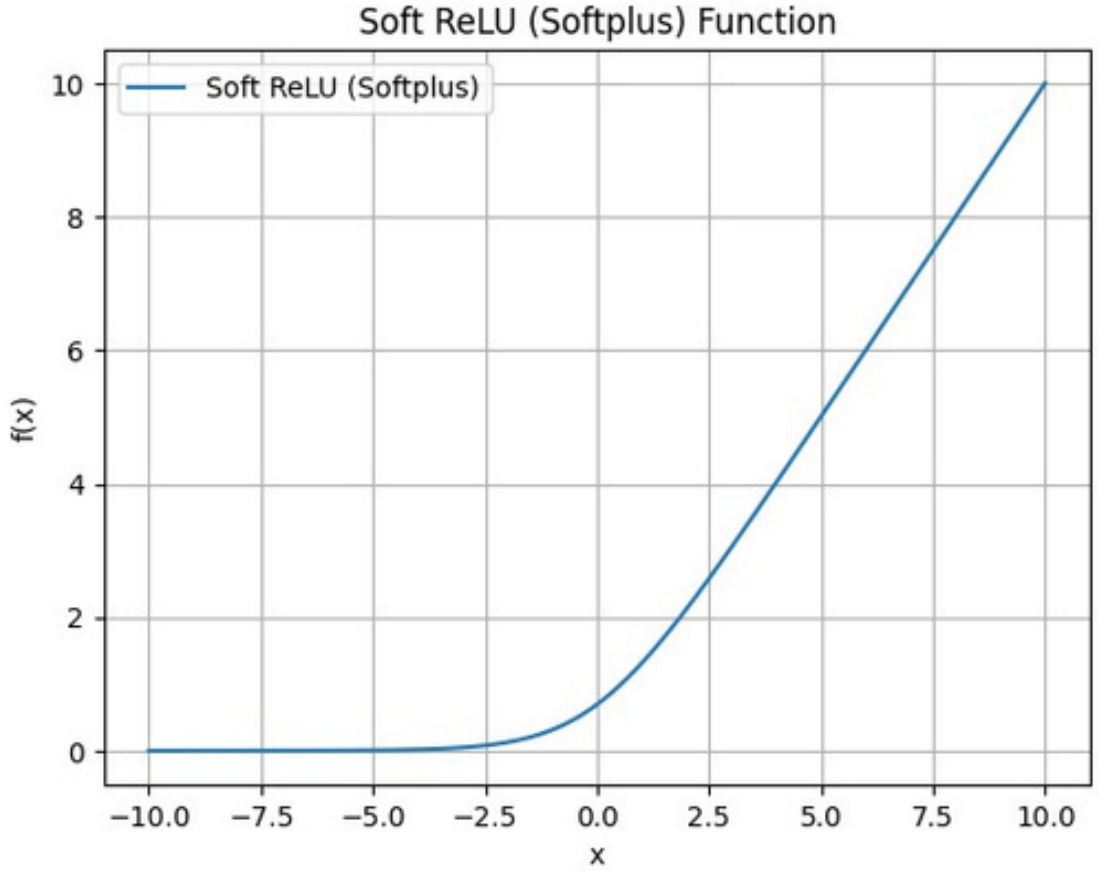
Activación de la unidad lineal rectificada (ReLU)



$$\sigma(z) = \max\{0, z\}$$

- Por defecto en la mayoría de redes feedforward
- Fácil de optimizar con métodos basados en gradientes
- Cero para entradas negativas
- No diferenciable en cero

Activación suave de ReLU (Softplus)



$$\sigma(z) = \log(1 + e^z)$$

- Aproxima ReLU
- Diferenciable en todas partes
- No produce gradientes cero para entradas negativas
- Computacionalmente costoso

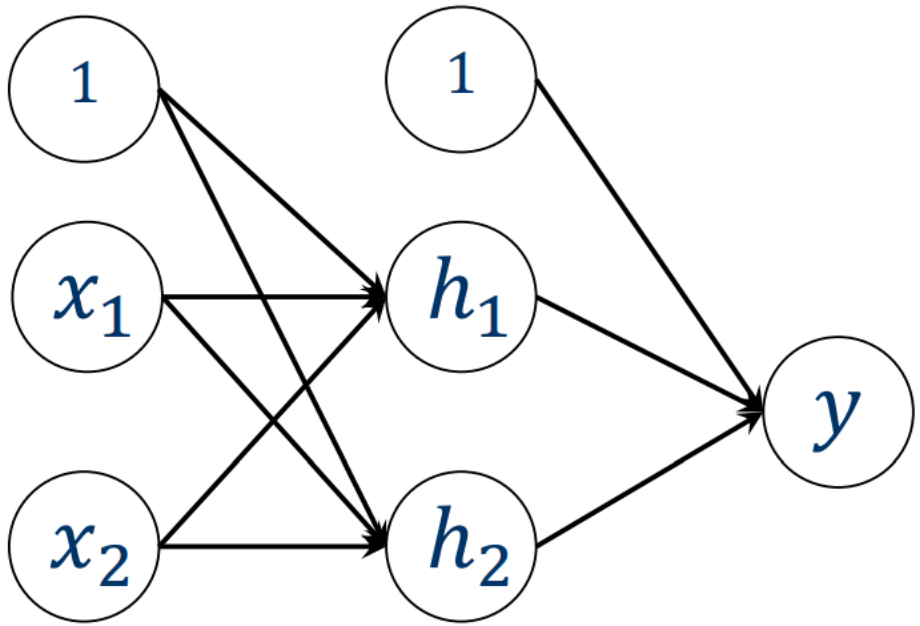
¿Cuál función utilizar y cuantas capas?
¿De qué depende?

Cálculo de XOR con una red neuronal

Ejercicio: Diseñe una red neuronal de 2 capas para calcular XOR.

(sólo utilizar pesos de 1,-1,-0.5)

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0



Utilice la activación del umbral en ambas capas.

$$f(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$

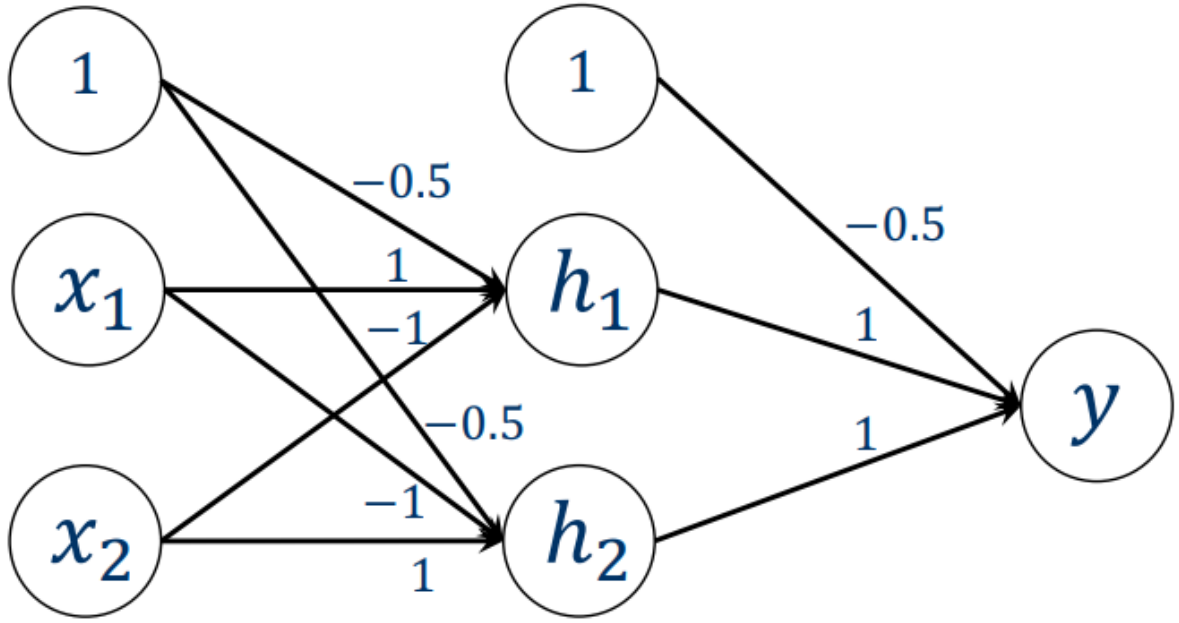
¿Cómo lo escribimos matricialmente?

Cálculo de XOR con una red neuronal

Solución 1:

$$x_1 \oplus x_2 = (x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge x_2)$$

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0



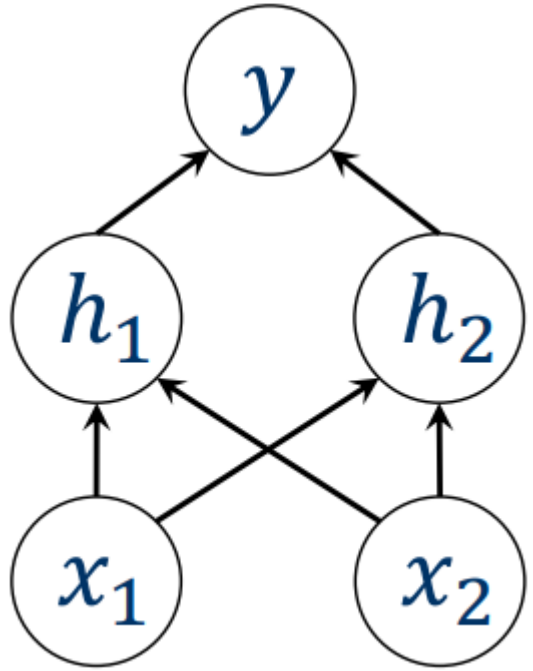
Ejercicio: Aprendiendo XOR

$$f(x) = w^T \max\{0, W^T x + c\} + b$$

$$W = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, c = \begin{bmatrix} 0 \\ -1 \end{bmatrix}$$

$$w = \begin{bmatrix} 1 \\ -2 \end{bmatrix}, b = 0$$

x_1	x_2	t
0	0	0
0	1	1
1	0	1
1	1	0



función de pérdida

$$J(\theta) = \frac{1}{4}\sum_{x}(y - t)^2$$

ReLU activation.

$$g(z) = \max\{0, z\}$$



Composición de funciones en conjunto

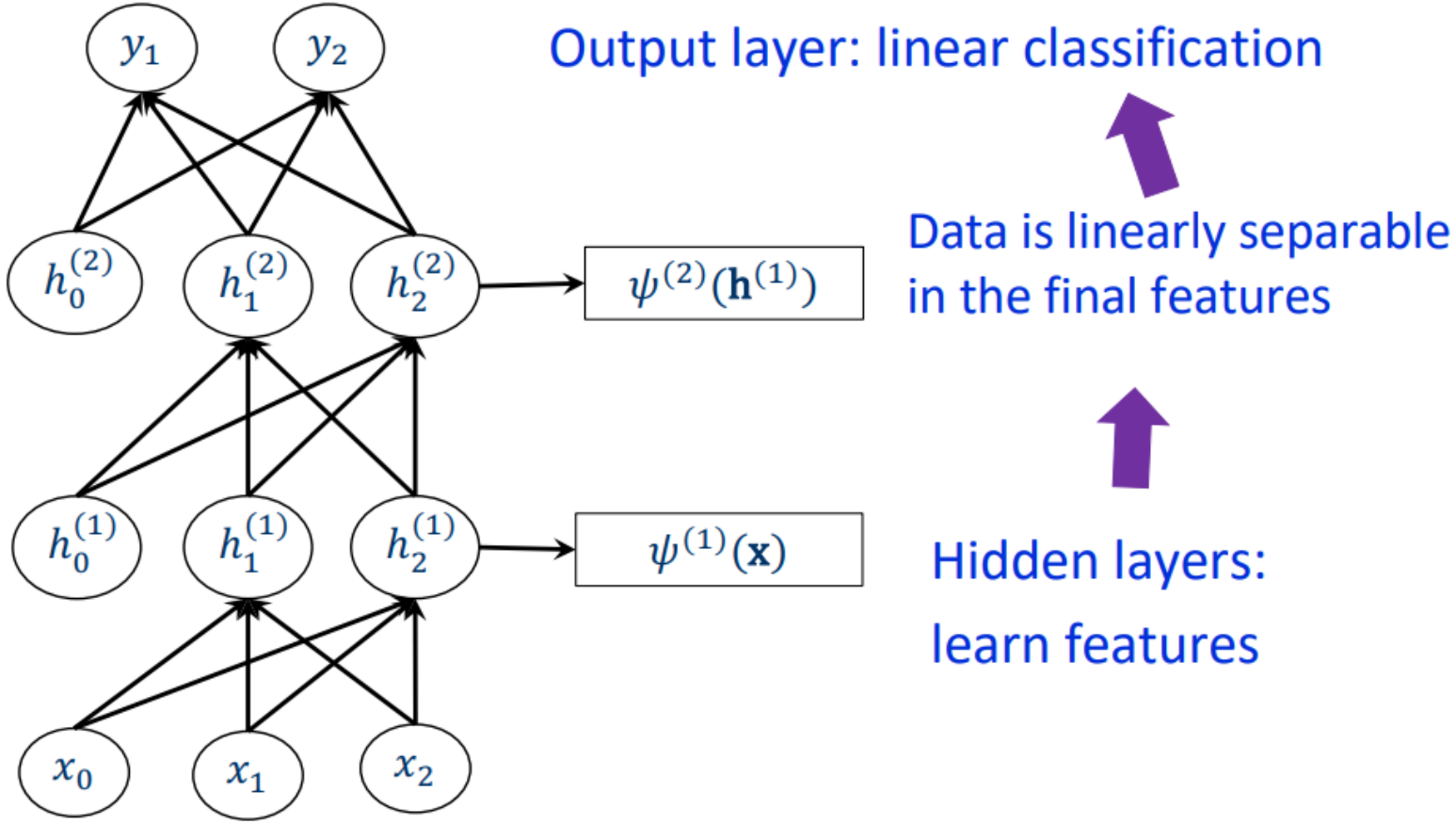
Cada capa calcula una función.

Una red calcula una composición de funciones.

Se pueden ver las redes neuronales como una forma de aprender características.

Aprendizaje de características

Se pueden ver las redes neuronales como una forma de aprender características.



¿Qué otro modelo hacia aprendizaje de caracteristicas?



Los poderes expresivos de las redes neuronales lineales

Los clasificadores lineales no pueden representar algunas funciones.

¿Son mejores las redes neuronales profundas?

Si todas las funciones de activación son lineales,

la red neuronal sólo puede representar funciones lineales.

--- no más expresiva que la regresión lineal.

Los poderes expresivos de las redes neuronales no lineales

Una red neuronal multicapa de propagación hacia adelante
con funciones de activación no lineales
es un aproximador **universal de funciones**.

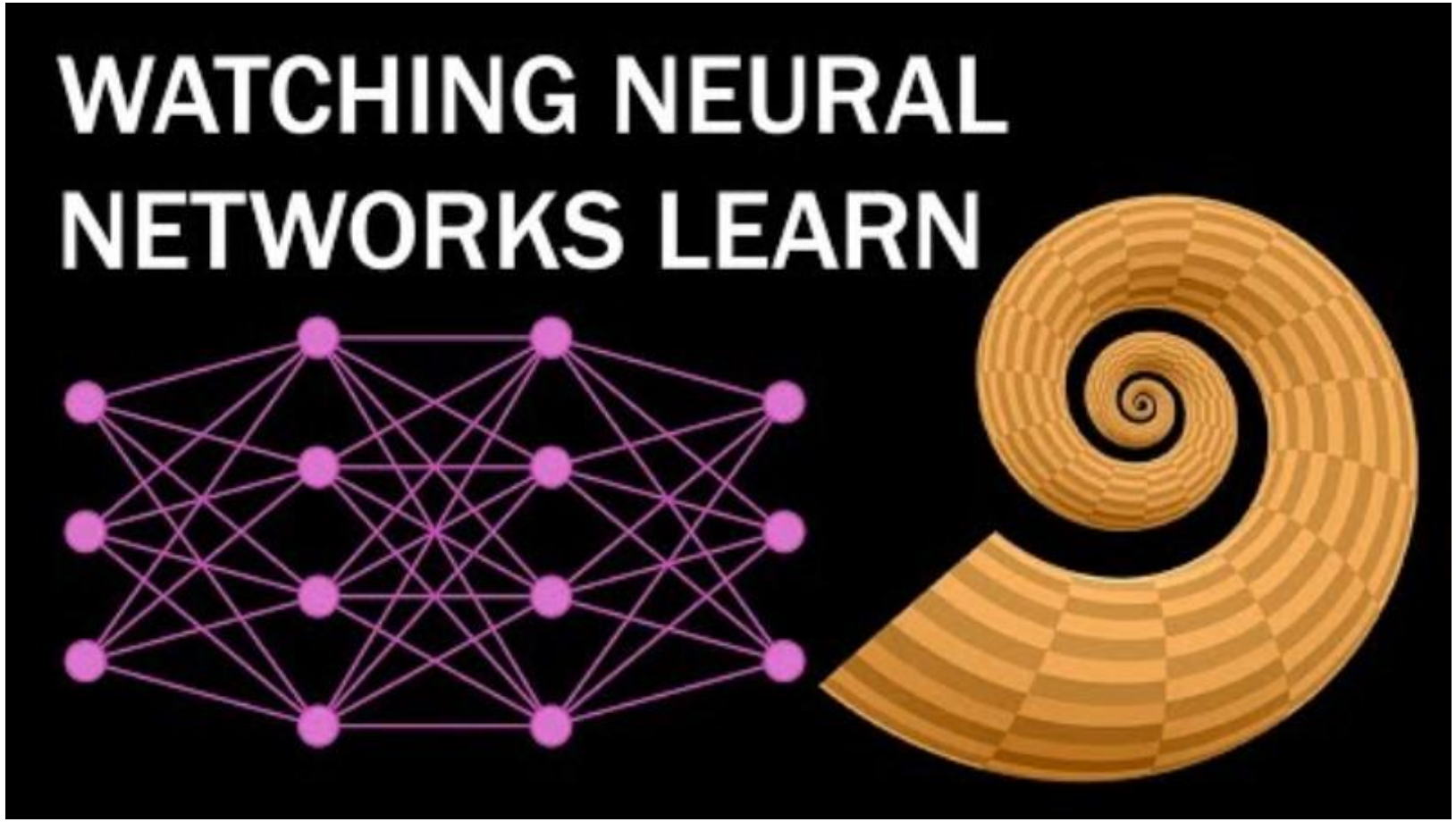
Pueden aproximar cualquier función arbitrariamente bien.

Verdadero para varias funciones de activación,
por ejemplo logística, ReLU, etc.

¿Qué otras funciones son aproximadores universales de funciones?

¿Sera una red neuronal mejor aproximación que una serie de Fourier o Taylor?

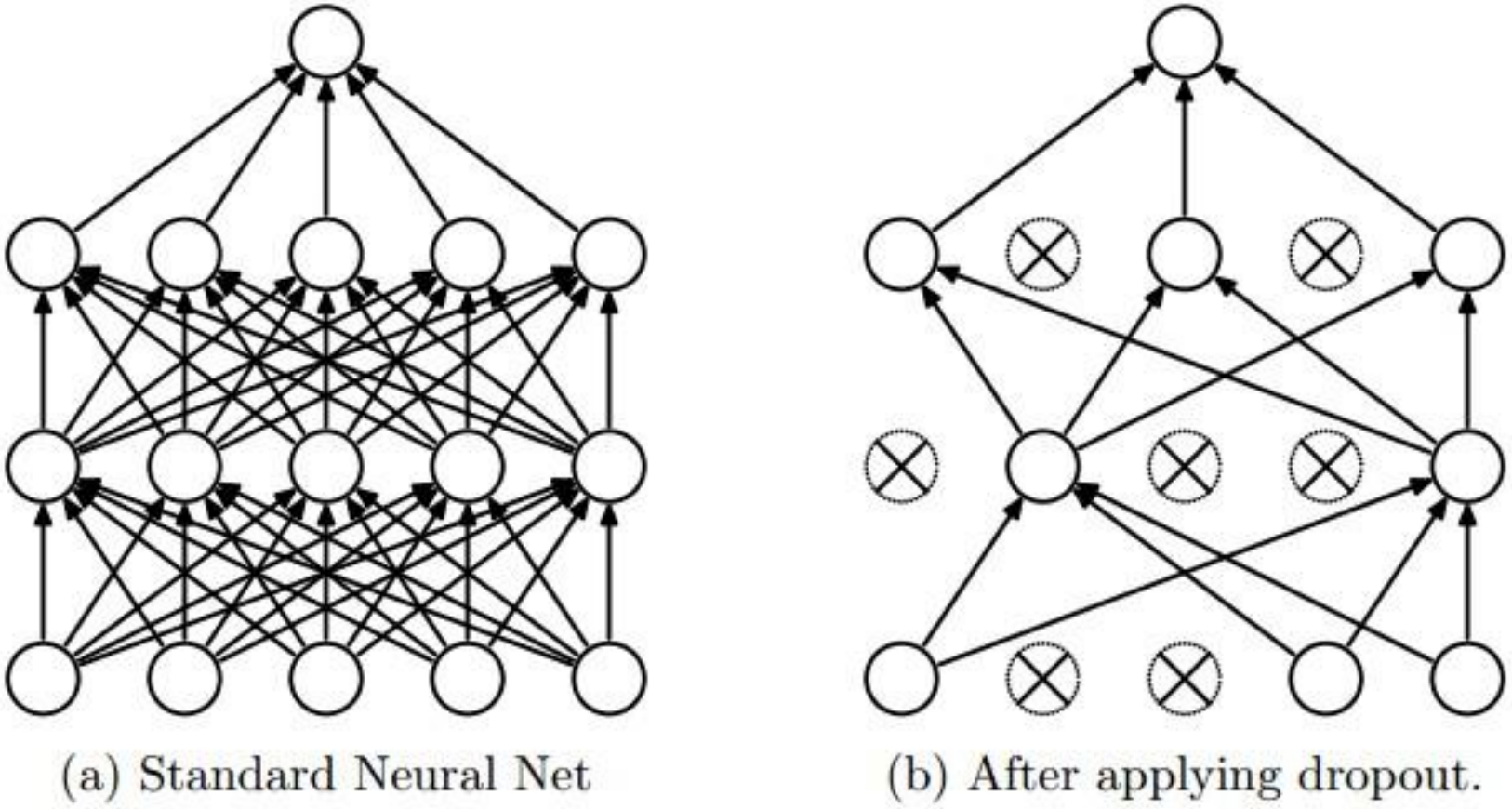
Watching Neural Networks Learn



https://youtu.be/TkwXa7Cvfr8?si=smcW9B_9zwRJ_B30&t=1286

¿Por qué la serie de Fourier sobre ajustaba y la red neuronal no?

Técnicas de Optimización: Dropout



(a) Standard Neural Net

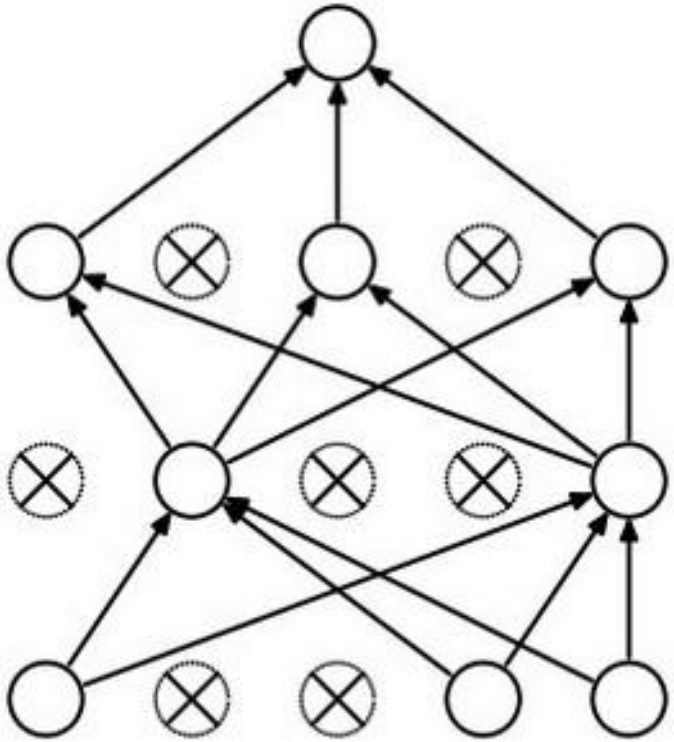
(b) After applying dropout.

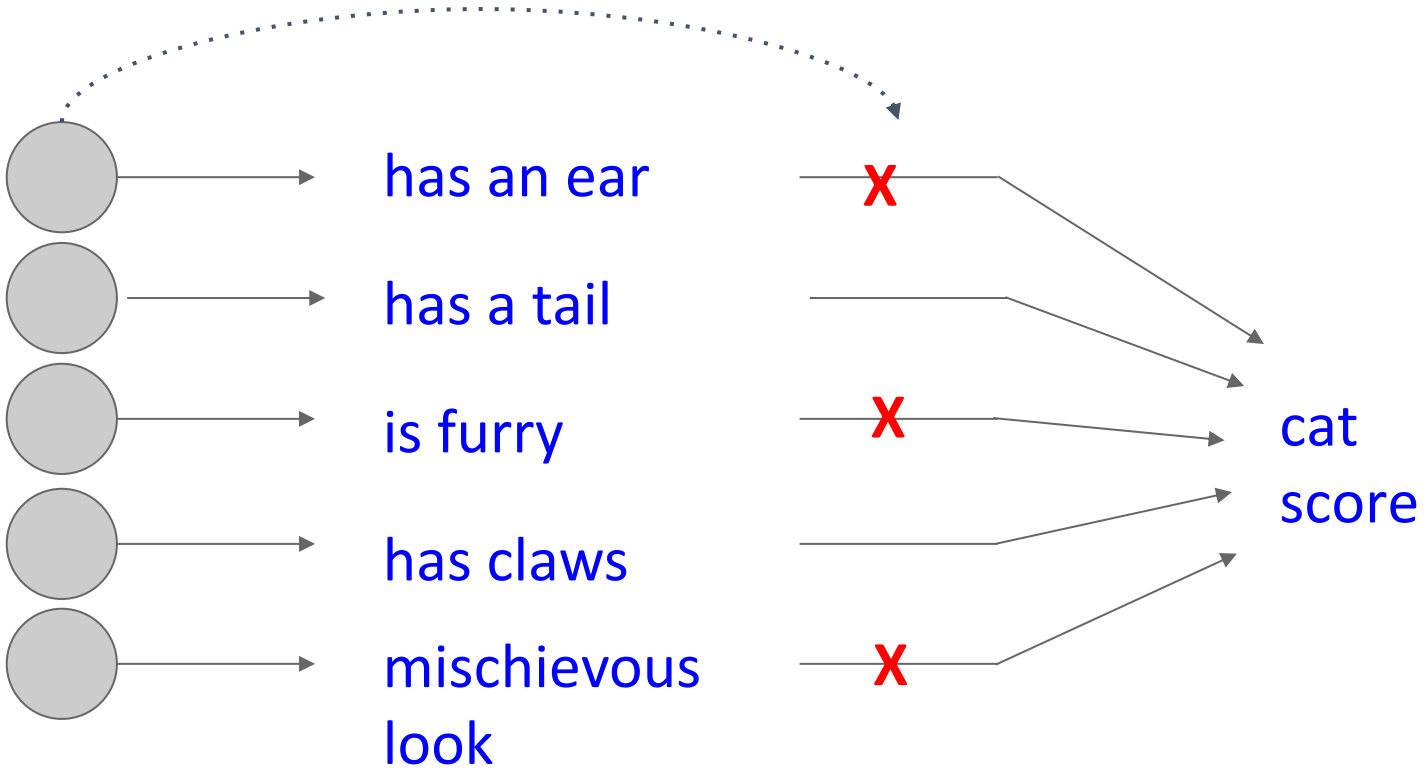
[Srivastava et al., 2014]

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

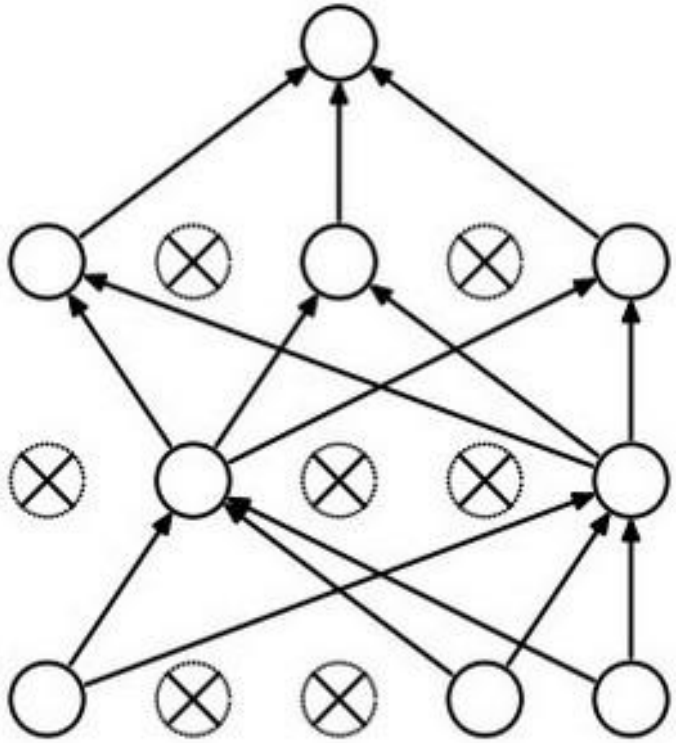
by Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov 2014

Técnicas de Optimización: Dropout





Técnicas de Optimización: Dropout



Se puede interpretar dropout como un ensamble.

La red va a observar un conjunto distinto de características en cada batch de entrenamiento.

Y para hacer las predicciones se usan todas las características.

(10 min) Break



Redes Neuronales: El Poder de la Multiplicación de Matrices

- Una red neuronal puede entenderse como una serie de multiplicaciones de matrices y aplicaciones de funciones no lineales.
- La multiplicación de matrices permite a la red combinar y transformar características de entrada de maneras complejas y no lineales.

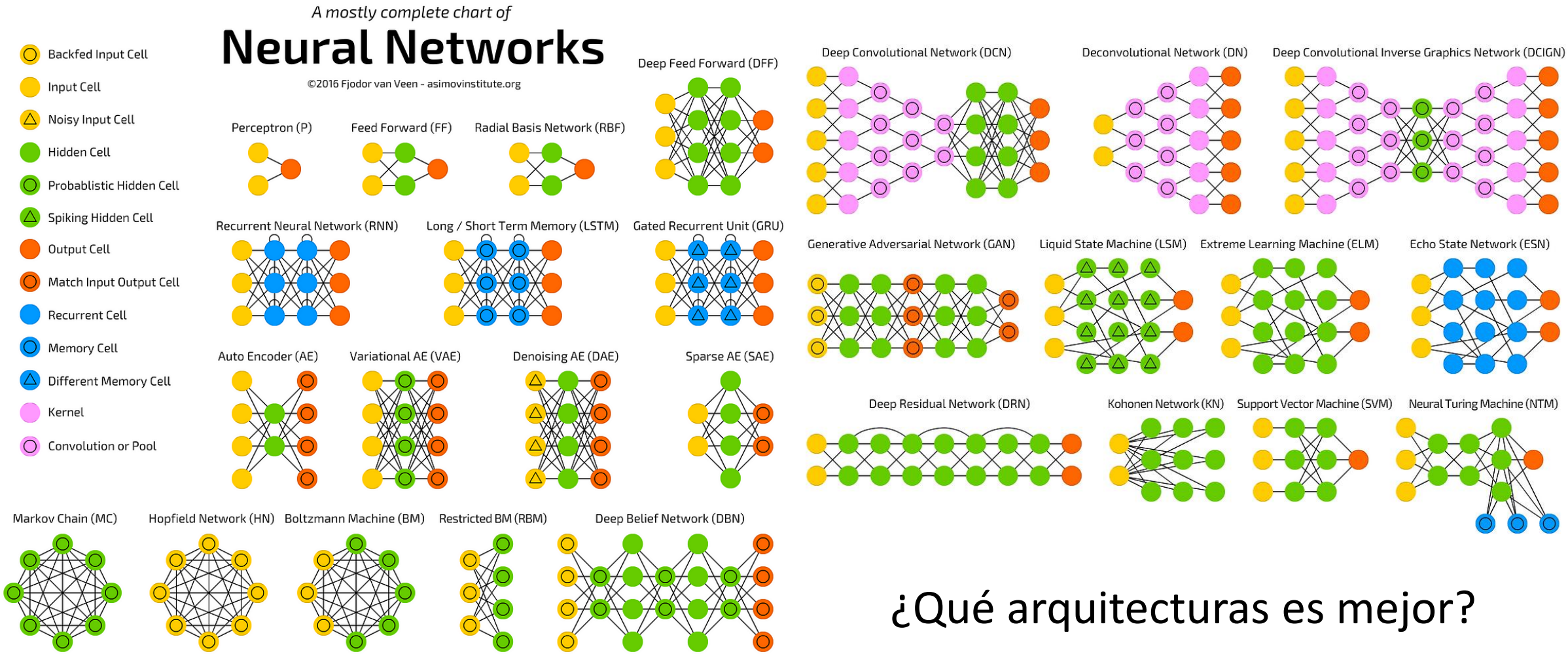
Grayscale Image



28 x 28

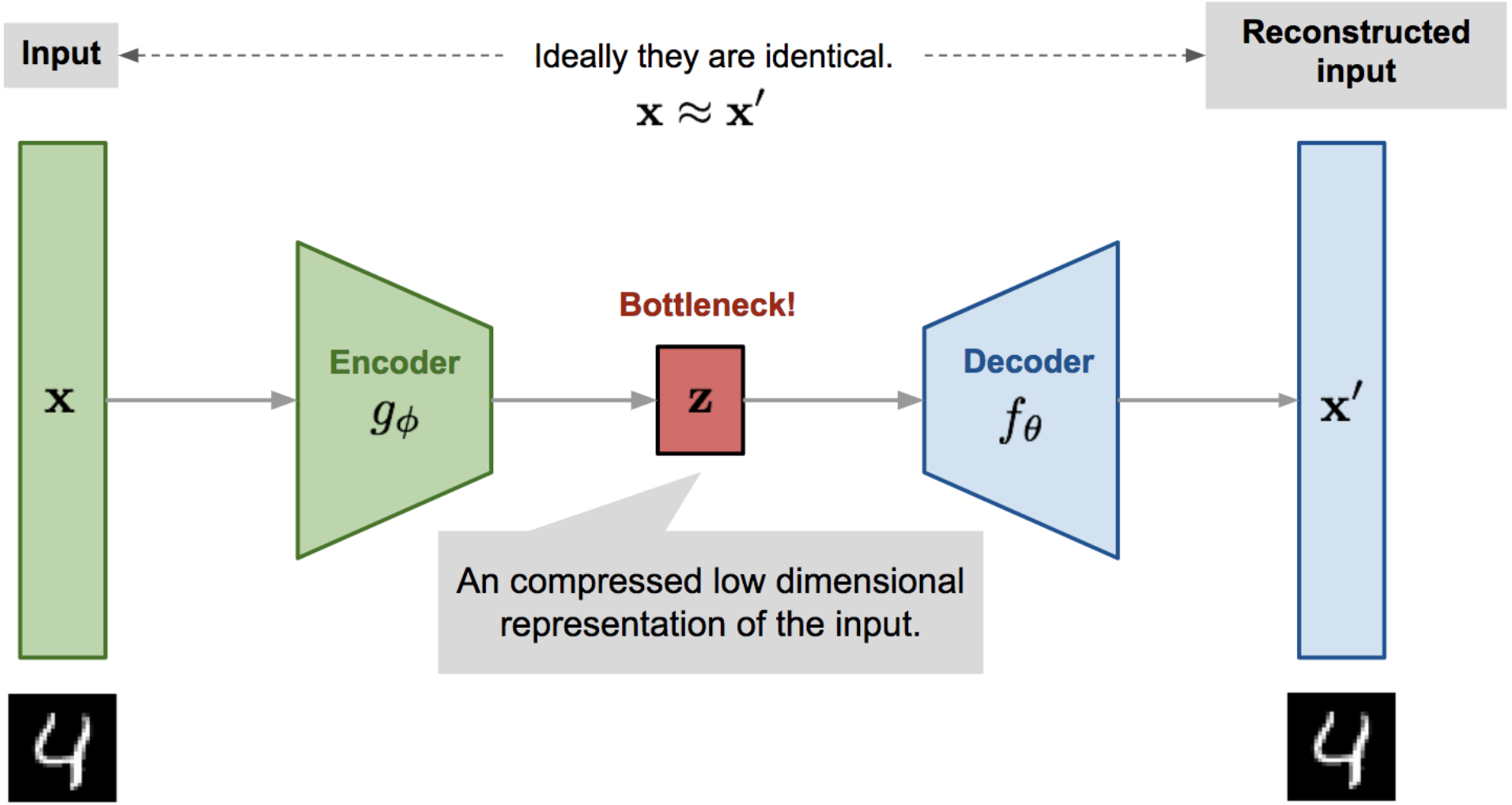
Pixel Intensities

```
[[  0   0   0   0   0   0   0   0   1  12  33  39  30   1   0   0   0   0]
 [  0   0   0   0   0   0   0   6  23   0   0   0   0  19   1   0   0   0]
 [  0   0   0   0   0   0   9   0   0  35 158 183 141   0   0   4   0   0]
 [  0   0   0   0   0  10   0  44 198 248 251 255 255 179   0  20   4   0]
 [  0   0   0   0   0  24   0 124 218 198 126  74 243 235  50   0  13   0]
 [  0   0   0   0   0   0   1  29   0   0   0 163 255 198   0  26   4   0]
 [  0   0   0   0   0   0   5   0   0   0 171 255 214  86   0  20   0   0]
 [  0   0   0   0   0   5   0 140 196 236 255 213   0   0  22   0   0   0]
 [  0   0   0   0  25   0 140 255 245 223 255 201   0  41   0   1   0   0]
 [  0   0   0   0   5   0  56  99  68   0 228 231  31   0  13   0   0   0]
 [  0   0   6  23   9   5   0   0  21   0 219 248  88   0  20   0   0   0]
 [  0   7   0   0   0  35  14  33  34   0 234 223   0  17   8   0   0   0]
 [ 21   0 118 178   0   0   0   0   0 174 255 145   0  24   1   0   0   0]
 [ 14   0  63 242 229 160  67  96 187 255 204   0  16   1   0   0   0   0]
 [  0  21   0 124 234 255 252 251 248 185   0  13   8   0   0   0   0   0]
 [  0   0  17   0   0 138 180 170  65   0  18   8   0   0   0   0   0   0]
 [  0   0   0  21  13   0   0   0   0  29   4   0   0   0   0   0   0   0]
 [  0   0   0   0   5  30  38  36  15   0   0   0   0   0   0   0   0   0]]
```

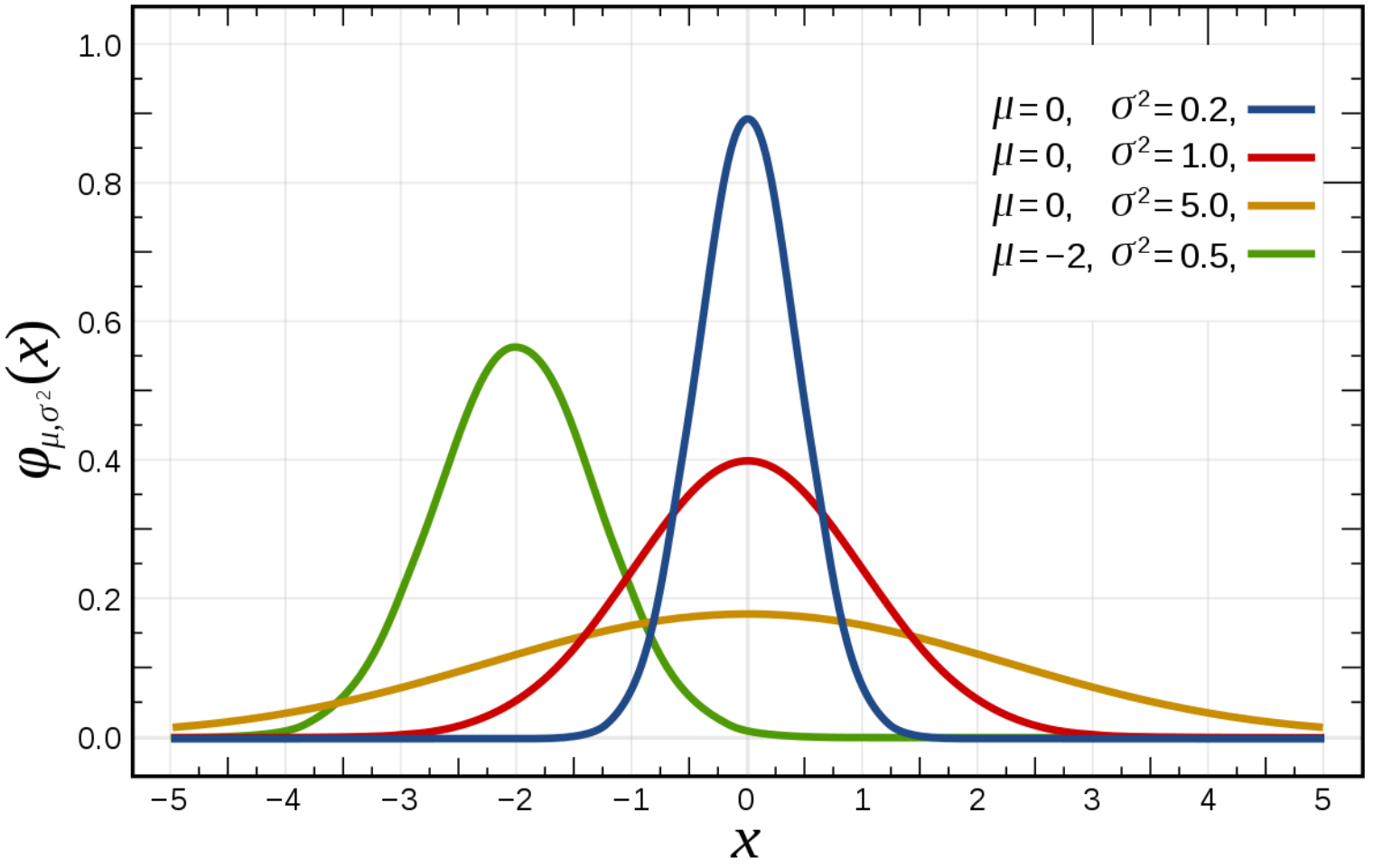


¿Qué arquitecturas es mejor?

Reducir la dimensionalidad: Autoencoders



En estadística es similar al concepto de distribución probabilística

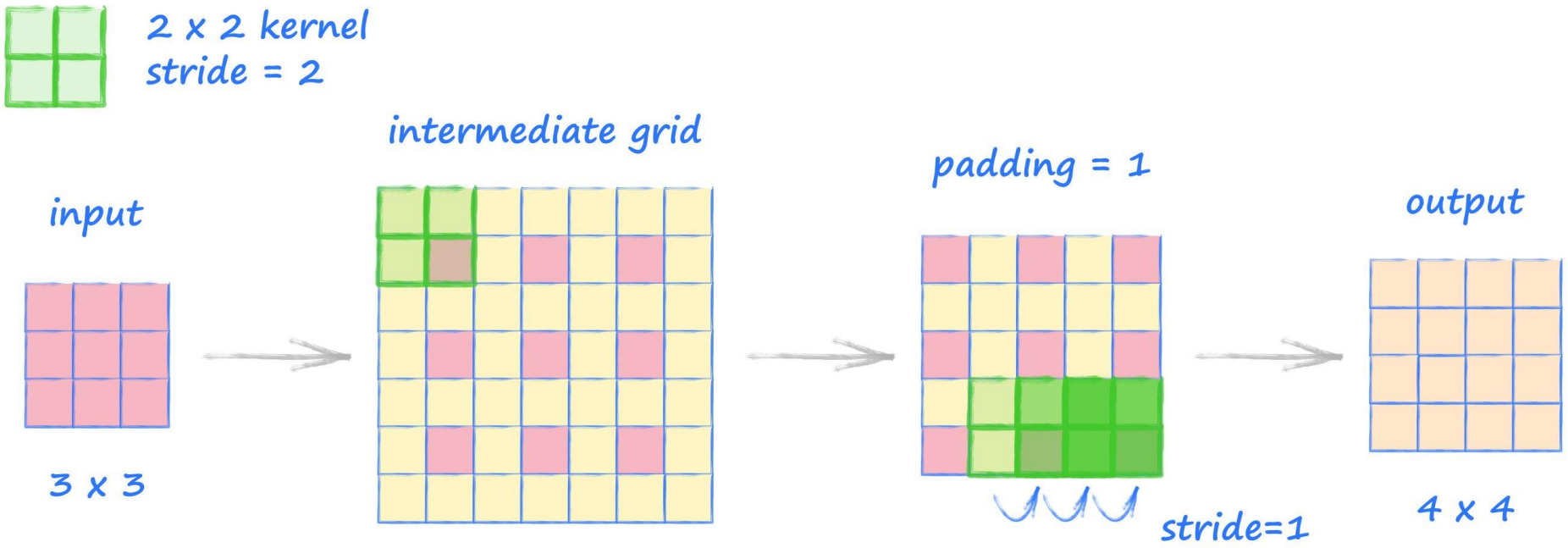


Funcionan bien para resolver problemas simples que no están influenciados por la escala



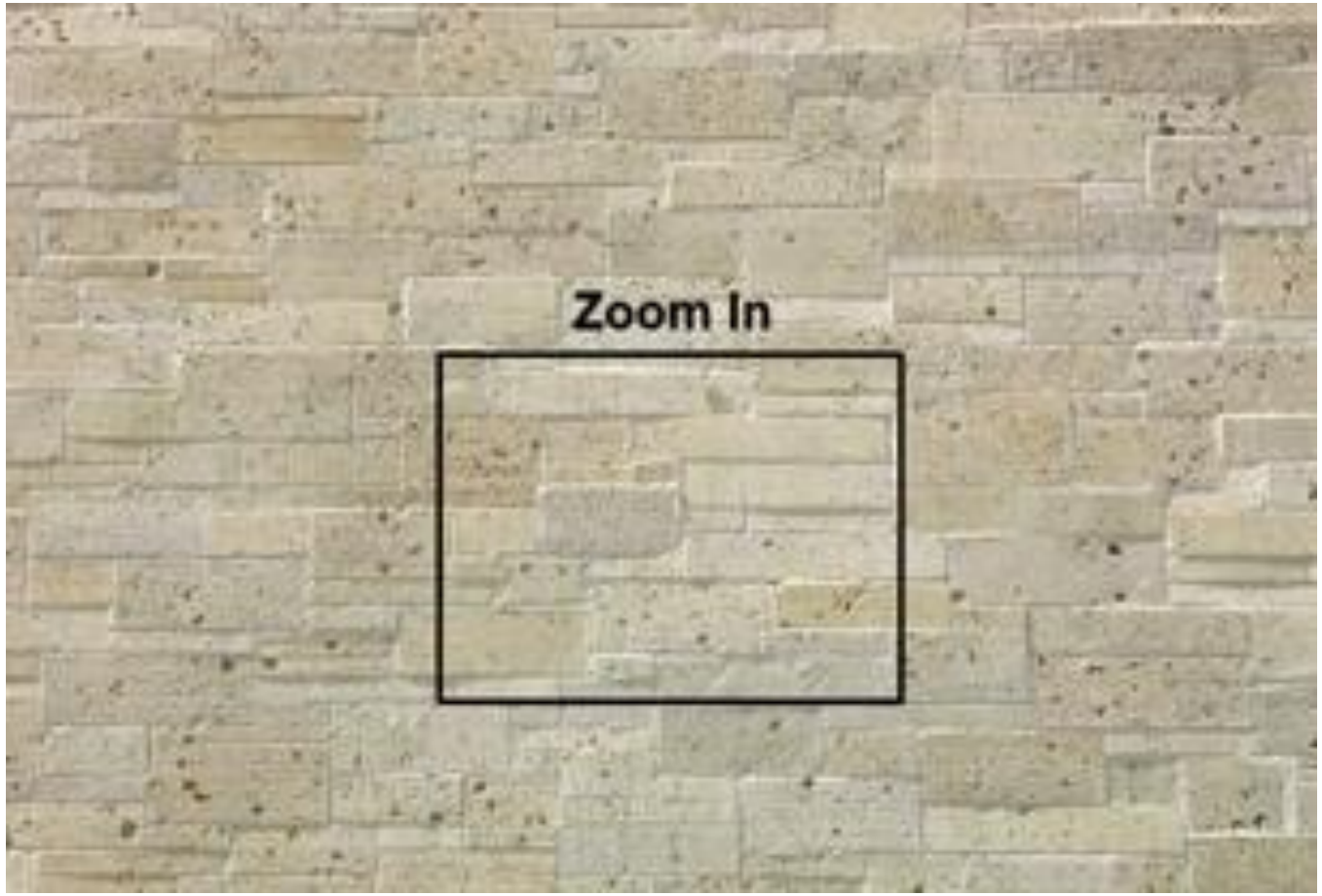
Redes Convolucionales

A distintas escalas se resuelven distintos problemas, estas redes se enfocan en relacionar o acotar el problema a distintas escalas.

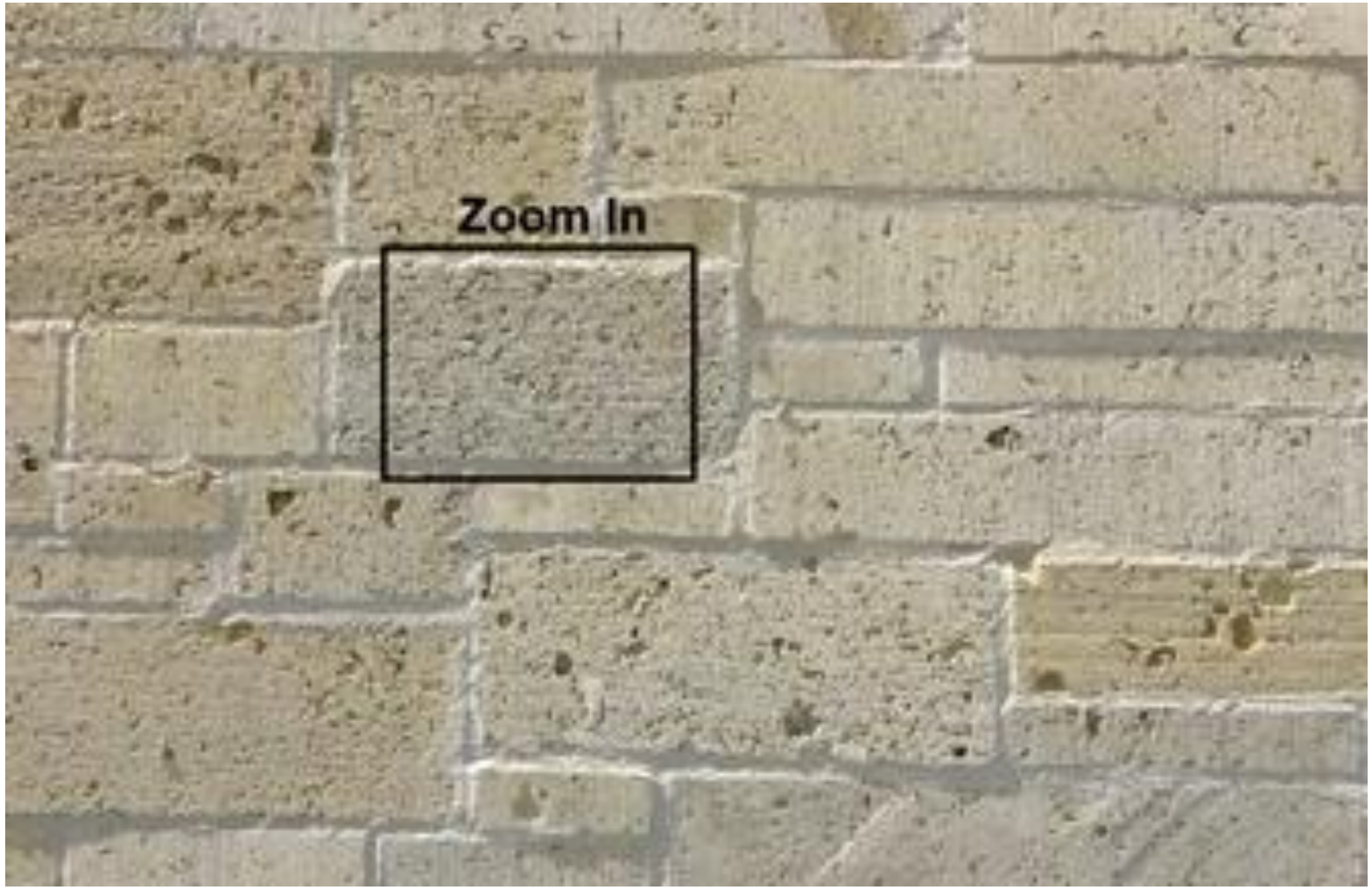


En geoestadística es similar al concepto de estacionariedad

- Cuando el promedio y la varianza se mantienen constantes en el espacio.
- ¿Esta serie de tiempo presenta estacionariedad?
- ¿Qué métricas usariamos?



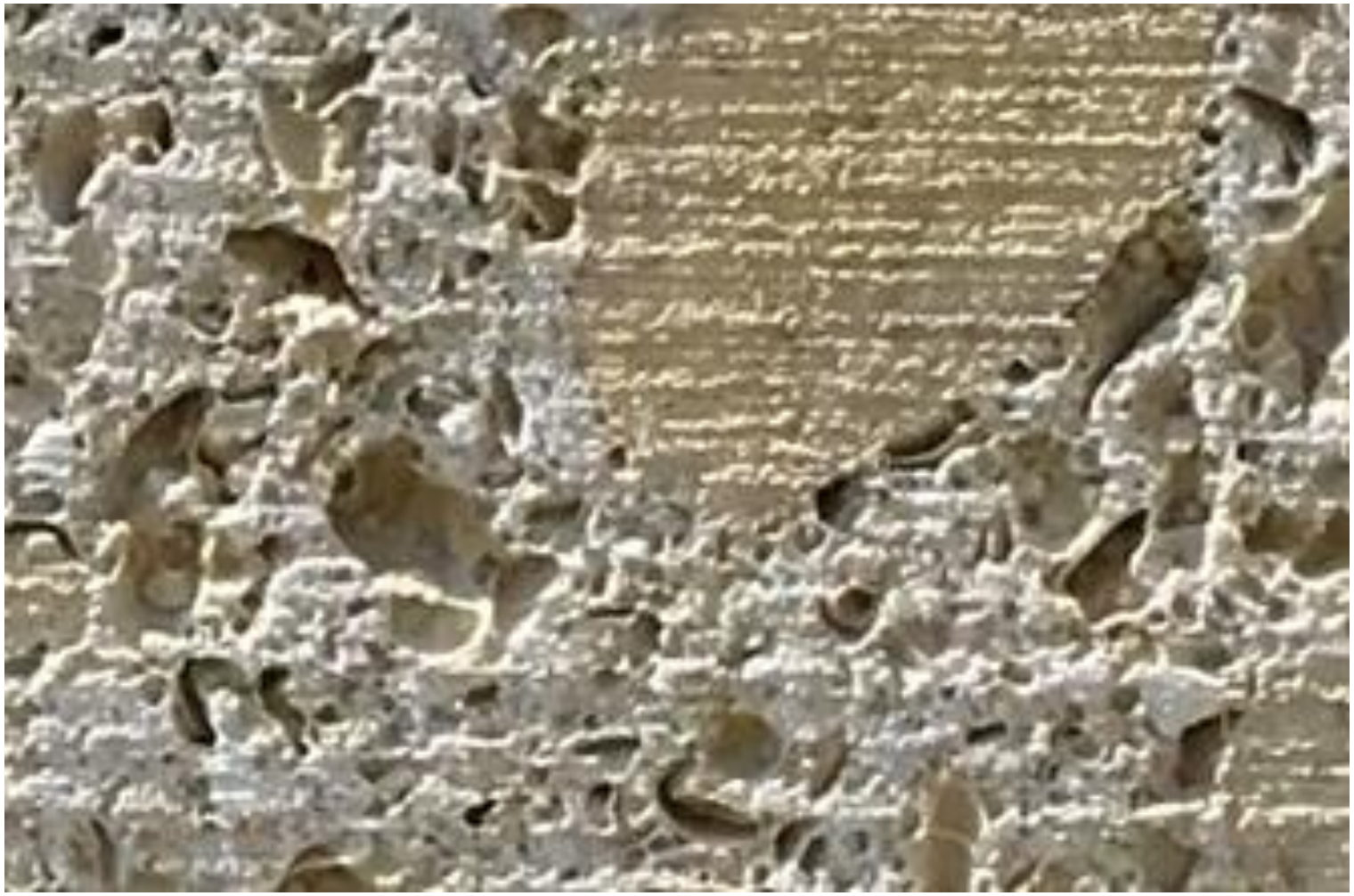
¿Es estacionario?



¿Es estacionario?



¿Es estacionario?

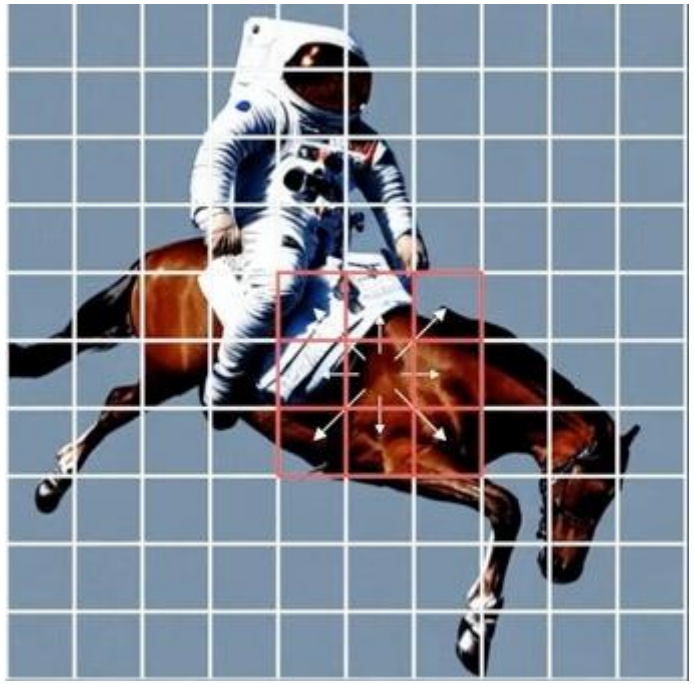


Cada escala tiene características particulares por lo que segmentar la imagen nos puede ayudar a capturar estas características.

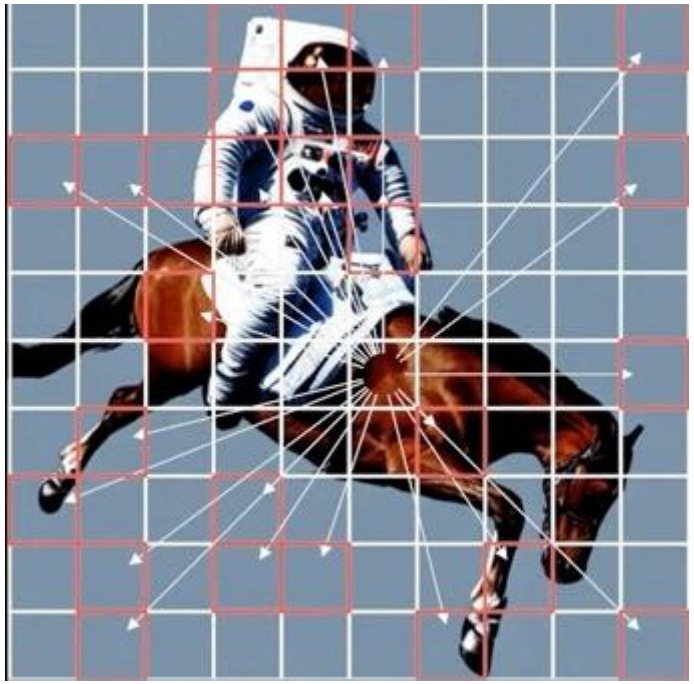
Transformers: Mejorando la Atención

En vez de enfocarse en ventanas específicas, la atención expande el enfoque en áreas particulares del dataset.

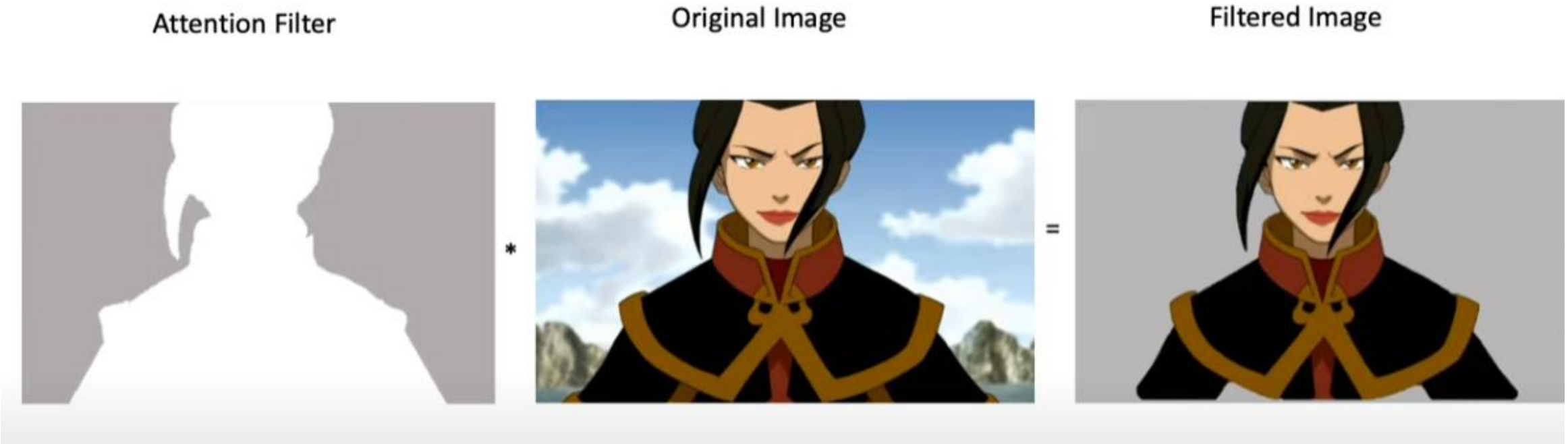
Convolución



Atención



La atención filtra la información necesaria a ser multiplicada



Visual Guide to Transformer Neural Networks - (Episode 2) Multi-Head & Self-Attention

https://www.youtube.com/watch?v=mMa2PmYJlCo

El Problema de encontrar los parámetros



“AI has thus far gotten to where it is via brute force”

Competencia por poder computacional

Para realizar muchas de las aplicaciones modernas de redes profundas es necesario poder computacional inmenso, creando una desventaja para países como Perú.



Problema: Detector de Picos

Considere una red neuronal con 1 neurona de entrada, 2 neuronas ocultas con activación ReLU, y 1 neurona de salida lineal. La red implementa la función:

$$f(x) = w^T \max\{ 0, W^T x + c \} + b; \text{ donde } W = [1, 1]^T;\ c = [0, -1]^T;\ w = [1, -2]^T;\ b = 0$$

El conjunto de entrenamiento tiene 3 ejemplos: (0,0), (1,1), (2,0). Y la función de pérdida es el error cuadrático medio.

Se pide:

a) Escriba explícitamente las funciones $h_1(x)$ y $h_2(x)$ de cada neurona oculta.

b) Realice el *forward pass* para cada ejemplo del conjunto de entrenamiento. Complete la tabla con los valores de h_1, h_2 e y.

c) Calcule la pérdida $J(\theta)$.

d) Explique qué "característica" extrae cada neurona oculta y por qué una sola neurona lineal no puede resolver este problema.

Solución (a)

Cada neurona oculta calcula $g(W_i^T x + c_i)$ donde g es la ReLU.
Sustituyendo los parámetros:

- $\mathbf{h_1(x)} = \max\{ 0, 1 \cdot x + 0 \} = \max\{ 0, x \} = \mathbf{ReLU(x)}$
- $\mathbf{h_2(x)} = \max\{ 0, 1 \cdot x + (-1) \} = \max\{ 0, x - 1 \} = \mathbf{ReLU(x - 1)}$

La salida de la red es: $y = 1 \cdot h_1 + (-2) \cdot h_2 + 0 = h_1 - 2h_2$

Solución (b)

Para x = 0:

- Preactivación: $W^T x + c = [0, -1]$
- Capa oculta: $h_1 = \text{ReLU}(0) = \mathbf{0}$ | $h_2 = \text{ReLU}(-1) = \mathbf{0}$
- Salida: $y = 0 - 2 \cdot 0 = \mathbf{0}$ (objetivo: t = 0) ✓

Para x = 1:

- Preactivación: $W^T x + c = [1, 0]$
- Capa oculta: $h_1 = \text{ReLU}(1) = \mathbf{1}$ | $h_2 = \text{ReLU}(0) = \mathbf{0}$
- Salida: $y = 1 - 2 \cdot 0 = \mathbf{1}$ (objetivo: t = 1) ✓

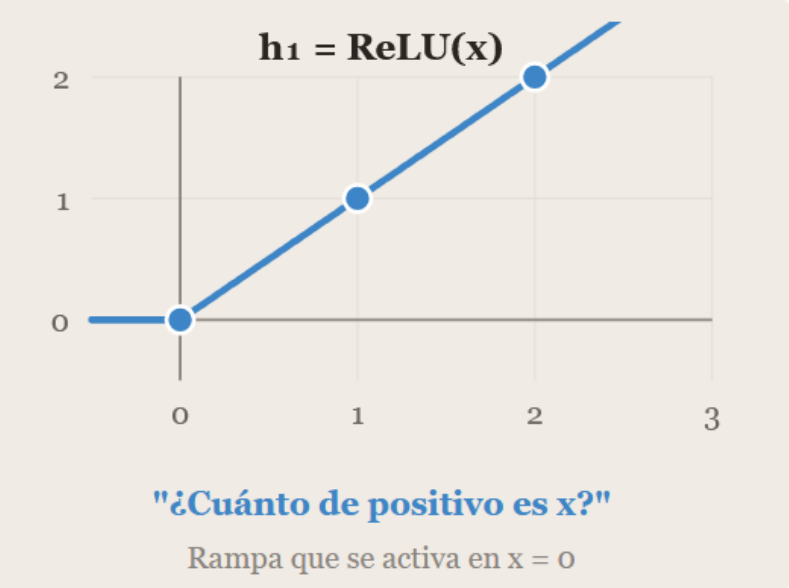
Para x = 2:

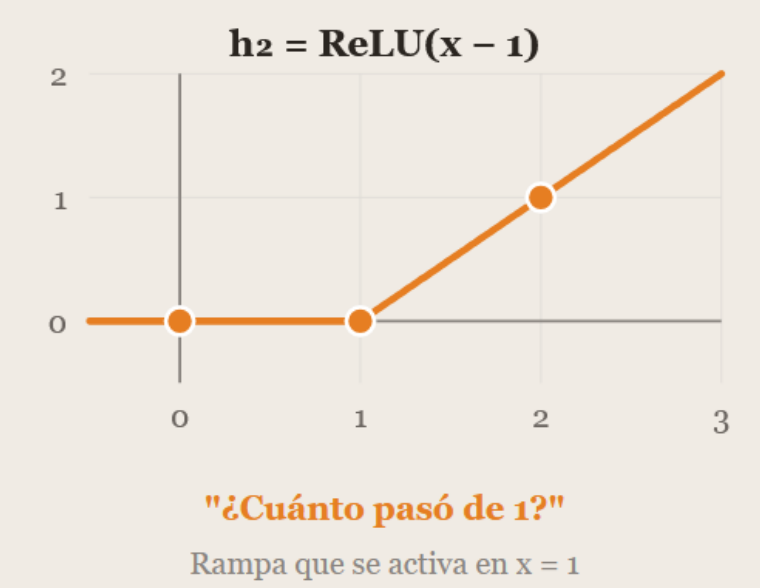
- Preactivación: $W^T x + c = [2, 1]$
- Capa oculta: $h_1 = \text{ReLU}(2) = \mathbf{2}$ | $h_2 = \text{ReLU}(1) = \mathbf{1}$
- Salida: $y = 2 - 2 \cdot 1 = \mathbf{0}$ (objetivo: t = 0) ✓

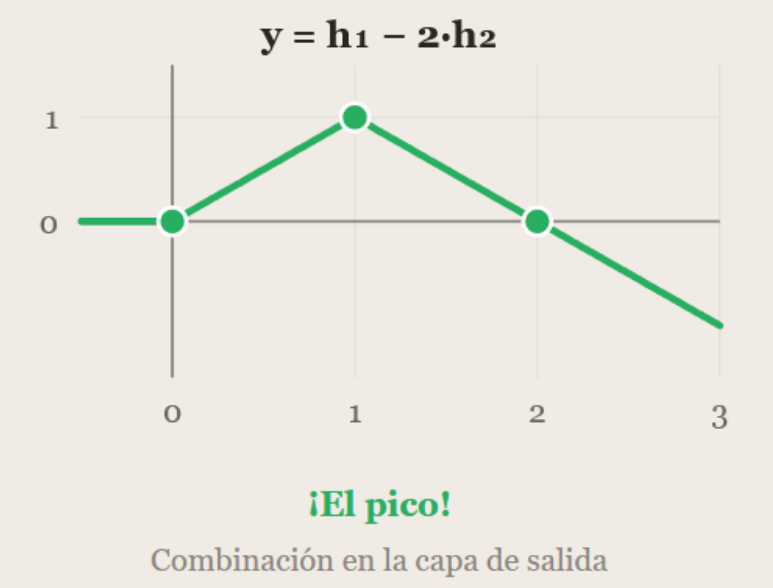
Solución (c)

- $J(\theta) = (1/3) \cdot [(0 - 0)^2 + (1 - 1)^2 + (0 - 0)^2]$
- $J(\theta) = (1/3) \cdot [0 + 0 + 0]$
- $J(\theta) = 0$

Solución (d)







h_1 detecta "x entró en la zona positiva" y h_2 detecta "x superó el umbral 1". La capa de salida suma h_1 (para subir al entrar en la región del pico) y resta $2 \times h_2$ (para bajar después del máximo). Cada neurona oculta es un simple detector de umbral; la combinación lineal en la capa de salida es lo que genera la forma no lineal.

Preguntas Vibe Quest

- Crea una herramienta interactiva que permita comparar el comportamiento de diferentes funciones de activación (ReLU, Sigmoid, Tanh, Leaky ReLU, ELU) en una red neuronal simple. La herramienta debe mostrar cómo cada función afecta el flujo del gradiente, la velocidad de convergencia y la capacidad de la red para aproximar funciones no lineales. Incluye visualizaciones del problema del 'vanishing gradient' y 'dead neurons'.
- Implementa una demostración interactiva del teorema de aproximación universal. Permite al usuario dibujar cualquier función continua en 2D y muestra cómo una red neuronal con diferentes arquitecturas (variando número de capas ocultas y neuronas) puede aproximarla. Visualiza cómo la complejidad de la red afecta la calidad de la aproximación y el riesgo de overfitting.“
- Crea una visualización que demuestre cómo el dropout actúa como una forma de ensemble learning. Implementa una red neuronal donde el usuario pueda ajustar la probabilidad de dropout y observar en tiempo real cómo esto afecta: (a) las predicciones individuales de diferentes 'sub-redes', (b) la incertidumbre en las predicciones, y (c) la capacidad de generalización. Compara el rendimiento con y sin dropout en datasets con diferentes niveles de ruido.“
- Desarrolla diferentes arquitecturas de redes profundas con el dataset MNIST o Fashion-MNIST. Explora que tipos de aprendizaje de características se pueden generar con distintas arquitecturas. ¿Qué patrones emergen en la representación aprendida?